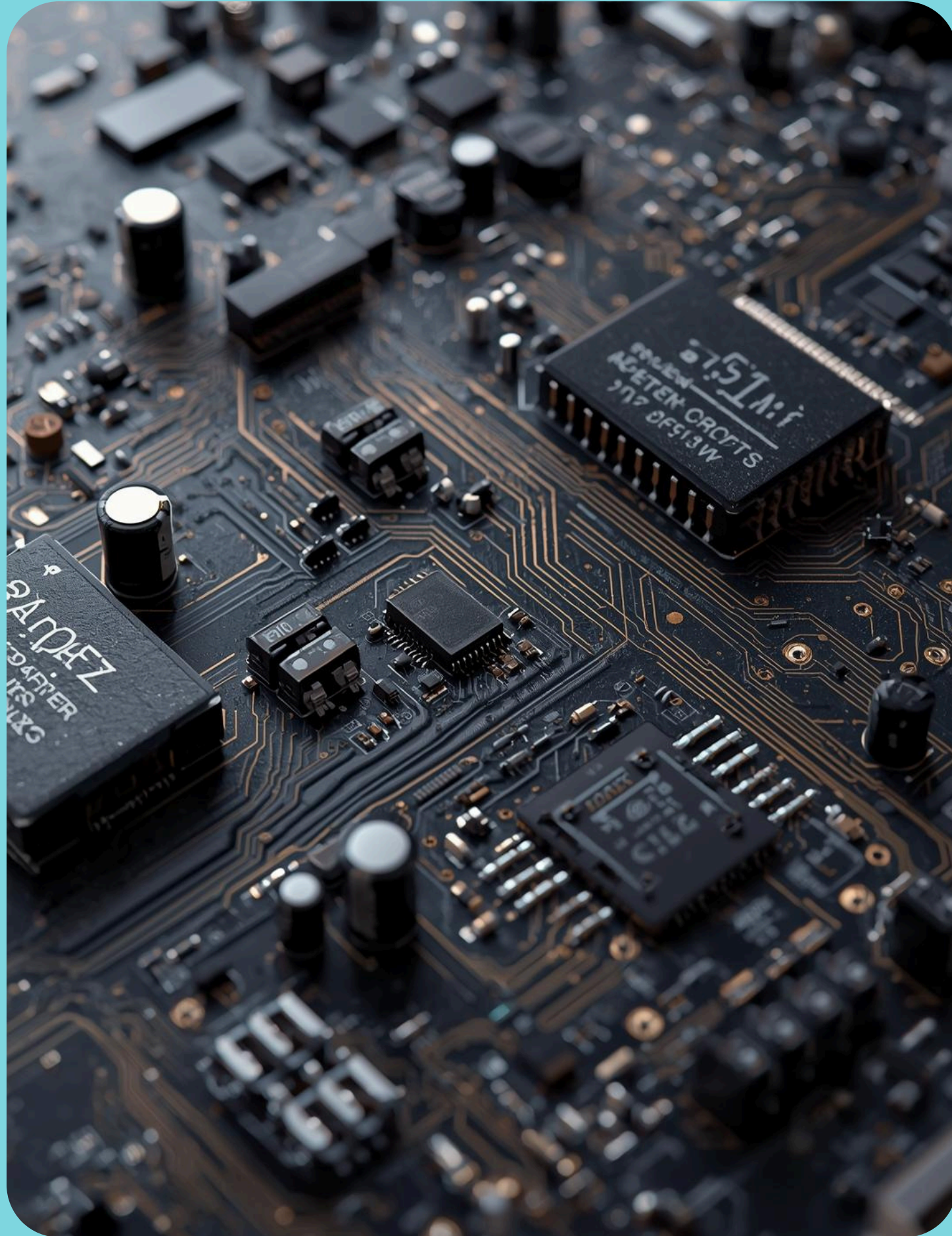




DESIGN AND SIMULATION OF AN 8-BIT RISC PROCESSOR USING VERILOG HDL AND FPGA BOARD

FINAL REVIEW

DSD PROJECT

MEMBERS

①

24BEC1407

V JAISHREE

②

24BEC1419

V G MANASA

③

24BEC1458

**SHRUTHI
NARAYANAN**

④

24BEC1540

JANANI S K



OBJECTIVE

- To Design a modular architecture 8-bit RISC CPU in Verilog HDL.
- To implementation of a basic instruction set.
- Creation of core blocks such as ALU, control unit, registers, program counter, instruction and data memory, and input/output interfaces.
- To use simulation tools such as ModelSim or FPGA board validation for verification.
- Demonstration of correct execution through waveform analysis or real board testing

METHODOLOGY

1. REQUIREMENT ANALYSIS

The 8-bit RISC processor requires essential hardware description modules to perform basic arithmetic and logical operations. The system must handle 8-bit wide data and support a minimal instruction set for operations such as addition, subtraction, logical AND, and OR. The design will include components such as the Arithmetic Logic Unit (ALU), register file, program counter, instruction memory, and control unit. The implementation will be carried out in Verilog HDL, simulated using appropriate software tools.

2. INSTRUCTION SET DESIGN

The instruction set for the 8-bit RISC processor is kept simple to ensure easy implementation and testing. It includes basic operations such as addition, subtraction, logical AND, logical OR, and data transfer between registers. Each instruction is represented by a unique opcode, which the control unit uses to identify and execute the required operation.



METHODOLOGY

3. ARCHITECTURE DESIGN

- Program Counter (PC) – Holds the address of the next instruction to be executed.
- Instruction Register (IR) – Stores the currently fetched instruction for decoding and execution.
- Arithmetic Logic Unit (ALU) – Performs arithmetic (addition, subtraction) and logical (AND, OR) operations.
- General Purpose Registers – Temporarily store data and intermediate results during program execution.
- Control Unit – Decodes the opcode and generates control signals to coordinate processor operations.
- Memory Interface – Manages data transfer between the processor and memory.

4. VERILOG IMPLEMENTATION

The processor is built in Verilog by creating separate modules for each part like the ALU, Program Counter, Registers, and Control Unit. Each module is connected to work together according to the instruction set. The opcodes are decoded to control the data flow and operations. Testbenches are used to check that each part and the whole processor work correctly in simulation.



METHODOLOGY

5. SIMULATION AND FUNCTIONAL VERIFICATION

- Simulation of all Verilog modules using ModelSim to validate functionality.
- Creating comprehensive testbenches covering all instructions and corner cases.
- Performance of waveform analysis to confirm correct timing, control signals, and data paths.
- Verifying proper execution of arithmetic, logic, and branch instructions.

6. DEBUGGING AND OPTIMIZATION

- Identifying and corrected functional mismatches using signal tracing.
- Refining RTL code to remove redundant logic and improve synthesis results.
- Optimising instruction execution speed and reduced critical path delays.
- Enhancing modularity for easier future upgrades.



METHODOLOGY

7. HARDWARE IMPLEMENTATION

- Synthesizing the verified RTL design using FPGA synthesis tools (Xilinx/Altera).
- Assigning FPGA pins according to the development board specifications.
- Generating bitstream and programmed it into the FPGA.
- Integrating clock, reset, and I/O interfaces for smooth hardware operation.

8. FINAL TESTING

- Executing predefined instruction sequences on FPGA hardware.
- Monitoring output on LEDs, seven-segment displays, or UART interface.
- Validating processor's performance under different clock speeds and load conditions.
- Confirming system stability and accuracy for prolonged operation.



TOOLS & PROGRAMMING LANGUAGES USED

1. HARDWARE DESCRIPTION LANGUAGE (HDL)

Verilog HDL that is used for designing the processor's data path, control unit, ALU, registers, etc.

2. SIMULATION TOOLS

ModelSim

Quartus Prime - for Intel FPGA boards.

3. HARDWARE

Intel/Altera FPGA boards (e.g., DE10-Lite).

Programming cables (JTAG/USB)



WORKFLOW

Designing the processor's parts—ALU, registers, control unit, and memory—in Verilog



Testing them using ModelSim



Converting the design into FPGA format using Quartus.



Loading this FPGA board and implementing it in the hardware





PARTS OF PROCESSOR

Program Counter



Instruction Memory



Control Unit



Arithmetic Logic Unit (ALU)



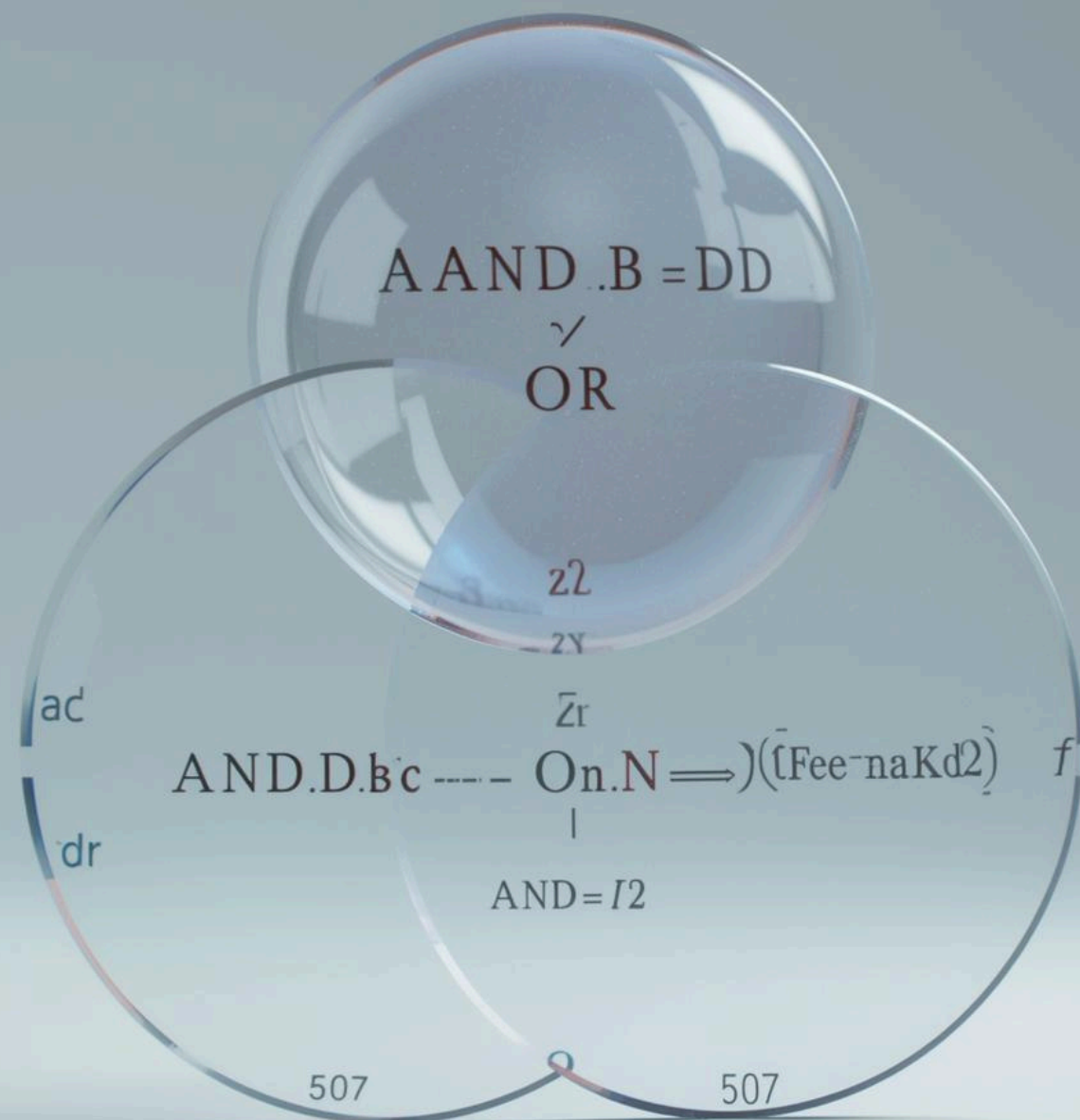
Register File



Data Memory



Output



VERILOG HDL CODE

VERILOG CODE FOR ADDITION

```
// =====  
// 8-BIT RISC PROCESSOR - SINGLE-CYCLE DESIGN  
// Target: Intel Cyclone V (5CSXFC6D6F31C6N)  
// Top module name: project  
// =====  
  
// Opcode Definitions (shared across modules)  
'define LDI 4'b0000 // Load Immediate (R0 <- Imm)  
'define STORE 4'b0001 // Store R0 to Memory (Mem[Imm] <- R0)  
'define ADD 4'b0010 // R0 <- R0 + R[Operand]  
'define SUB 4'b0011 // R0 <- R0 - R[Operand]  
'define AND 4'b0100  
'define OR 4'b0101  
'define XOR 4'b0110  
'define JMP 4'b0111  
'define JZ 4'b1000  
'define HALT 4'b1111
```

```
// =====  
// 1. PROGRAM COUNTER MODULE  
// =====  
module program_counter(  
    input clk,  
    input reset,  
    input load_pc,  
    input pc_enable,  
    input [7:0] next_pc_address,  
    output reg [7:0] pc_out  
);  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            pc_out <= 8'h00;  
        else if (load_pc)  
            pc_out <= next_pc_address;  
        else if (pc_enable)  
            pc_out <= pc_out + 1;  
        end  
    endmodule
```

```
// =====  
// 2. INSTRUCTION MEMORY MODULE  
// =====  
module instruction_memory(  
    input wire [7:0] address,  
    output reg [7:0] instruction  
);  
    reg [7:0] mem [0:255];  
    integer i;  
    initial begin  
        // Program: Display 4, then 6, then 4+6=10  
        // Step 1: Load 4 into R0, HALT to display  
        mem[8'h00] = {`LDI, 4'h4}; // R0 = 4  
        mem[8'h01] = {`HALT, 4'h0}; // Halt to display 4  
  
        // Step 2: Load 6 into R0, HALT to display  
        mem[8'h02] = {`LDI, 4'h6}; // R0 = 6  
        mem[8'h03] = {`HALT, 4'h0}; // Halt to display 6  
  
        // Step 3: Load 4, ADD 6 from R1, HALT to display result  
        mem[8'h04] = {`LDI, 4'h4}; // R0 = 4  
        mem[8'h05] = {`ADD, 4'h1}; // R0 = R0 + R1 (R1=6, so R0=10)  
        mem[8'h06] = {`HALT, 4'h0}; // Halt to display 10  
  
        for (i = 7; i < 256; i = i + 1)  
            mem[i] = 8'h00;  
        end  
  
        always @(*) begin  
            instruction = mem[address];  
        end  
    endmodule
```

```
input wire [7:0] address,  
output reg [7:0] instruction  
);  
    reg [7:0] mem [0:255];  
    integer i;  
    initial begin  
        // Program: Display 4, then 6, then 4+6=10  
        // Step 1: Load 4 into R0, HALT to display  
        mem[8'h00] = {`LDI, 4'h4}; // R0 = 4  
        mem[8'h01] = {`HALT, 4'h0}; // Halt to display 4  
  
        // Step 2: Load 6 into R0, HALT to display  
        mem[8'h02] = {`LDI, 4'h6}; // R0 = 6  
        mem[8'h03] = {`HALT, 4'h0}; // Halt to display 6  
  
        // Step 3: Load 4, ADD 6 from R1, HALT to display result  
        mem[8'h04] = {`LDI, 4'h4}; // R0 = 4  
        mem[8'h05] = {`ADD, 4'h1}; // R0 = R0 + R1 (R1=6, so R0=10)  
        mem[8'h06] = {`HALT, 4'h0}; // Halt to display 10  
  
        for (i = 7; i < 256; i = i + 1)  
            mem[i] = 8'h00;  
        end  
  
        always @(*) begin  
            instruction = mem[address];  
        end  
    endmodule
```

```
// =====  
// 3. REGISTER FILE MODULE  
// =====  
module register_file(  
    input wire clk,  
    input wire reset,  
    input wire reg_write,  
    input wire [2:0] read_addr1,  
    input wire [2:0] read_addr2,  
    input wire [2:0] write_addr,  
    input wire [7:0] write_data,  
    output reg [7:0] read_data1,  
    output reg [7:0] read_data2  
);  
    reg [7:0] registers [0:7];  
    integer i;  
    initial begin  
        for (i = 0; i < 8; i = i + 1)  
            registers[i] = 8'h00;  
        registers[3'h1] = 8'h06; // R1 = 6 (for addition)  
        registers[3'h2] = 8'h04; // R2 = 4 (optional)  
    end  
  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            for (i = 0; i < 8; i = i + 1)  
                registers[i] <= 8'h00;  
            registers[3'h1] <= 8'h06; // R1 = 6  
            registers[3'h2] <= 8'h04; // R2 = 4  
        end else if (reg_write && write_addr == 3'b000)  
            registers[write_addr] <= write_data;  
        end  
endmodule
```

```
integer i;  
  
initial begin  
    for (i = 0; i < 8; i = i + 1)  
        registers[i] = 8'h00;  
        registers[3'h1] = 8'h06; // R1 = 6 (for addition)  
        registers[3'h2] = 8'h04; // R2 = 4 (optional)  
    end  
  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            for (i = 0; i < 8; i = i + 1)  
                registers[i] <= 8'h00;  
            registers[3'h1] <= 8'h06; // R1 = 6  
            registers[3'h2] <= 8'h04; // R2 = 4  
        end else if (reg_write && write_addr == 3'b000)  
            registers[write_addr] <= write_data;  
        end  
    end  
endmodule
```

```
// =====  
// 4. ALU MODULE  
// =====  
module alu(  
    input wire [7:0] operand_a,  
    input wire [7:0] operand_b,  
    input wire [3:0] alu_control,  
    output reg [7:0] alu_result,  
    output reg zero_flag  
);  
    always @(*) begin  
        case (alu_control)  
            `ADD: alu_result = operand_a + operand_b;  
            `SUB: alu_result = operand_a - operand_b;  
            `AND: alu_result = operand_a & operand_b;  
            `OR: alu_result = operand_a | operand_b;  
            `XOR: alu_result = operand_a ^ operand_b;  
            default: alu_result = operand_a;  
        endcase  
    end  
endmodule
```

```
initial begin  
    for (i = 0; i < 256; i = i + 1)  
        mem[i] = 8'h00;  
    end  
    always @(posedge clk)  
        if (mem_write) mem[address] <= write_data;  
    always @(*) begin  
        if (mem_read) read_data = mem[address];  
        else read_data = 8'h00;  
    end  
endmodule  
  
// =====  
// 6. CONTROL UNIT MODULE  
// =====  
module control_unit(  
    input wire [3:0] opcode,  
    input wire zero_flag,  
    output reg pc_write_cu,  
    output reg reg_write,  
    output reg mem_write,  
    output reg mem_read,  
    output reg [1:0] reg_write_src,  
    output reg [3:0] alu_control,  
    output reg halt  
);  
    parameter WRITE_ALU = 2'b00, WRITE_MEM = 2'b01, WRITE_IMM = 2'b10;  
    always @(*) begin  
        pc_write_cu = 1'b1;  
        reg_write = 1'b0;  
        mem_write = 1'b0;  
        mem_read = 1'b0;  
        reg_write_src = WRITE_ALU;  
        alu_control = `ADD;  
        halt = 1'b0;  
        case (opcode)  
            `LDI: begin reg_write = 1'b1; reg_write_src = WRITE_IMM; end  
            `STORE: begin mem_write = 1'b1; end  
            `ADD, `SUB, `AND, `OR, `XOR:  
                begin reg_write = 1'b1; alu_control = opcode; reg_write_src = WRITE_ALU; end  
            `JMP: pc_write_cu = 1'b1;  
            `JZ: pc_write_cu = 1'b1;  
            `HALT: begin halt = 1'b1; pc_write_cu = 1'b0; end  
        endcase  
    end  
endmodule
```

```
end  
endmodule  
  
// =====  
// 7. MAIN RISC PROCESSOR CORE  
// =====  
module risc_processor_8bit(  
    input wire clk,  
    input wire reset,  
    output wire [7:0] pc_out,  
    output wire [7:0] acc_out,  
    output wire halt_flag  
);  
    wire [7:0] pc_current, instruction, reg_data1, reg_data2, alu_result, mem_data,  
    reg_write_data;  
    wire [3:0] opcode = instruction[7:4];  
    wire [3:0] operand = instruction[3:0];  
    wire zero_flag, pc_write_cu, reg_write, mem_write, mem_read, halt;  
    wire [1:0] reg_write_src;  
    wire [3:0] alu_control;  
  
    wire load_pc_jump = (opcode == `JMP) || (opcode == `JZ && zero_flag);  
    wire [7:0] jump_address = {4'h0, operand};  
    wire pc_clk = clk; // Always run PC with system clock  
  
    assign reg_write_data = (reg_write_src == 2'b10) ? {4'h0, operand} :  
        (reg_write_src == 2'b01) ? mem_data : alu_result;  
  
    assign pc_out = pc_current;  
    assign acc_out = RF.registers[0];  
    assign halt_flag = halt;  
  
    program_counter PC (.clk(pc_clk),  
        .reset(reset),  
        .load_pc(load_pc_jump),  
        .pc_enable(pc_write_cu), // <--- connect control signal  
        .next_pc_address(jump_address),  
        .pc_out(pc_current)  
    );  
    instruction_memory IMEM (.address(pc_current), .instruction(instruction));  
    register_file RF (.clk(clk), .reset(reset), .reg_write(reg_write),  
        .read_addr1(3'b000), .read_addr2(operand[2:0]),  
        .write_addr(3'b000), .write_data(reg_write_data),  
    );  
endmodule
```

```

    end
endmodule

// =====
// 7. MAIN RISC PROCESSOR CORE
// =====
module risc_processor_8bit(
    input wire clk,
    input wire reset,
    output wire [7:0] pc_out,
    output wire [7:0] acc_out,
    output wire halt_flag
);
    wire [7:0] pc_current, instruction, reg_data1, reg_data2, alu_result, mem_data,
    reg_write_data;
    wire [3:0] opcode = instruction[7:4];
    wire [3:0] operand = instruction[3:0];
    wire zero_flag, pc_write_cu, reg_write, mem_write, mem_read, halt;
    wire [1:0] reg_write_src;
    wire [3:0] alu_control;

    wire load_pc_jump = (opcode == `JMP) || (opcode == `JZ && zero_flag);
    wire [7:0] jump_address = {4'h0, operand};
    wire pc_clk = clk; // Always run PC with system clock

    assign reg_write_data = (reg_write_src == 2'b10) ? {4'h0, operand} :
        (reg_write_src == 2'b01) ? mem_data : alu_result;

    assign pc_out = pc_current;
    assign acc_out = RF.registers[0];
    assign halt_flag = halt;

    program_counter PC (.clk(pc_clk),
        .reset(reset),
        .load_pc(load_pc_jump),
        .pc_enable(pc_write_cu), // <--- connect control signal
        .next_pc_address(jump_address),
        .pc_out(pc_current)
    );

    instruction_memory IMEM (.address(pc_current), .instruction(instruction));
    register_file RF (.clk(clk), .reset(reset), .reg_write(reg_write),
        .read_addr1(3'b000), .read_addr2(operand[2:0]),
        .write_addr(3'b000), .write_data(reg_write_data),

```

```

        .read_data1(reg_data1), .read_data2(reg_data2));
    alu ALU (.operand_a(reg_data1), .operand_b(reg_data2),
        .alu_control(alu_control), .alu_result(alu_result), .zero_flag(zero_flag));
    data_memory DMEM (.clk(clk), .mem_write(mem_write), .mem_read(mem_read),
        .address({4'h0, operand}), .write_data(reg_data1), .read_data(mem_data));
    control_unit CU (.opcode(opcode), .zero_flag(zero_flag), .pc_write_cu(pc_write_cu),
        .reg_write(reg_write), .mem_write(mem_write), .mem_read(mem_read),
        .reg_write_src(reg_write_src), .alu_control(alu_control), .halt(halt));
endmodule

// =====
// 8. HEX DECODER
// =====
module hex7seg(
    input [3:0] nibble,
    output reg [6:0] seg
);
    always @(*) begin
        case (nibble)
            4'h0: seg = 7'b1000000;
            4'h1: seg = 7'b1111001;
            4'h2: seg = 7'b0100100;
            4'h3: seg = 7'b0110000;
            4'h4: seg = 7'b0011001;
            4'h5: seg = 7'b0010010;
            4'h6: seg = 7'b0000010;
            4'h7: seg = 7'b1111000;
            4'h8: seg = 7'b0000000;
            4'h9: seg = 7'b0010000;
            4'hA: seg = 7'b0001000;
            4'hB: seg = 7'b0000011;
            4'hC: seg = 7'b1000110;
            4'hD: seg = 7'b0100001;
            4'hE: seg = 7'b0000110;
            4'hF: seg = 7'b0001110;
        endcase
    end
endmodule

// =====
// 11. BINARY TO DECIMAL CONVERTER (0–99)
// =====
module bin_to_dec (
    input [7:0] binary,
    output reg [3:0] tens,

```

```

    output reg [3:0] ones
);
    integer value;
    always @(*) begin
        value = binary;
        tens = (value / 10) % 10;
        ones = value % 10;
    end
endmodule

// =====
// 9. FPGA WRAPPER FOR DE10-STANDARD BOARD
// =====
module project (
    input wire    CLOCK_50, // 50 MHz system clock
    input wire [3:0] KEY,    // Push-buttons (KEY0 active-low reset)
    input wire [9:0] SW,     // Slide switches
    output wire [9:0] LED,   // LEDs
    output wire [6:0] HEX0,
    output wire [6:0] HEX1
);
    wire clk = CLOCK_50;
    // Slow clock divider for visual debug (~3 Hz)
    reg [23:0] slow;
    always @(posedge CLOCK_50)
        slow <= slow + 1;
    wire slow_clk = slow[23];
    wire reset = ~KEY[0];

    wire [7:0] pc_out, acc_out;
    wire halt_flag;

    risc_processor_8bit CPU (
        .clk(slow_clk),
        .reset(reset),
        .pc_out(pc_out),
        .acc_out(acc_out),
        .halt_flag(halt_flag)
    );

    assign LED[7:0] = pc_out;
    assign LED[9:8] = {halt_flag, reset};
    wire [3:0] tens_digit, ones_digit;

```

```

    bin_to_dec DEC (.binary(acc_out), .tens(tens_digit), .ones(ones_digit));

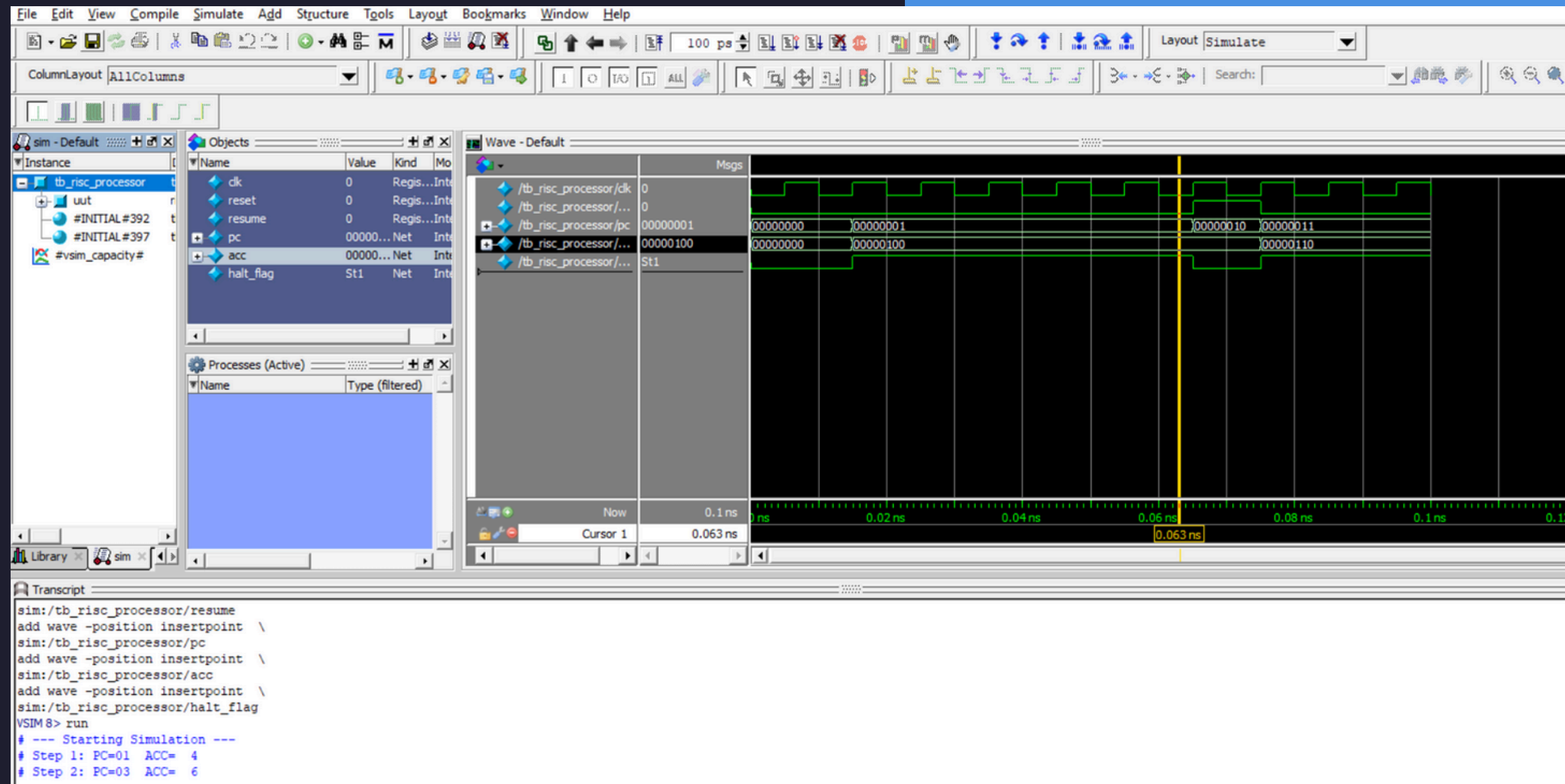
    hex7seg h0 (.nibble(ones_digit), .seg(HEX0)); // Ones place
    hex7seg h1 (.nibble(tens_digit), .seg(HEX1)); // Tens place

endmodule

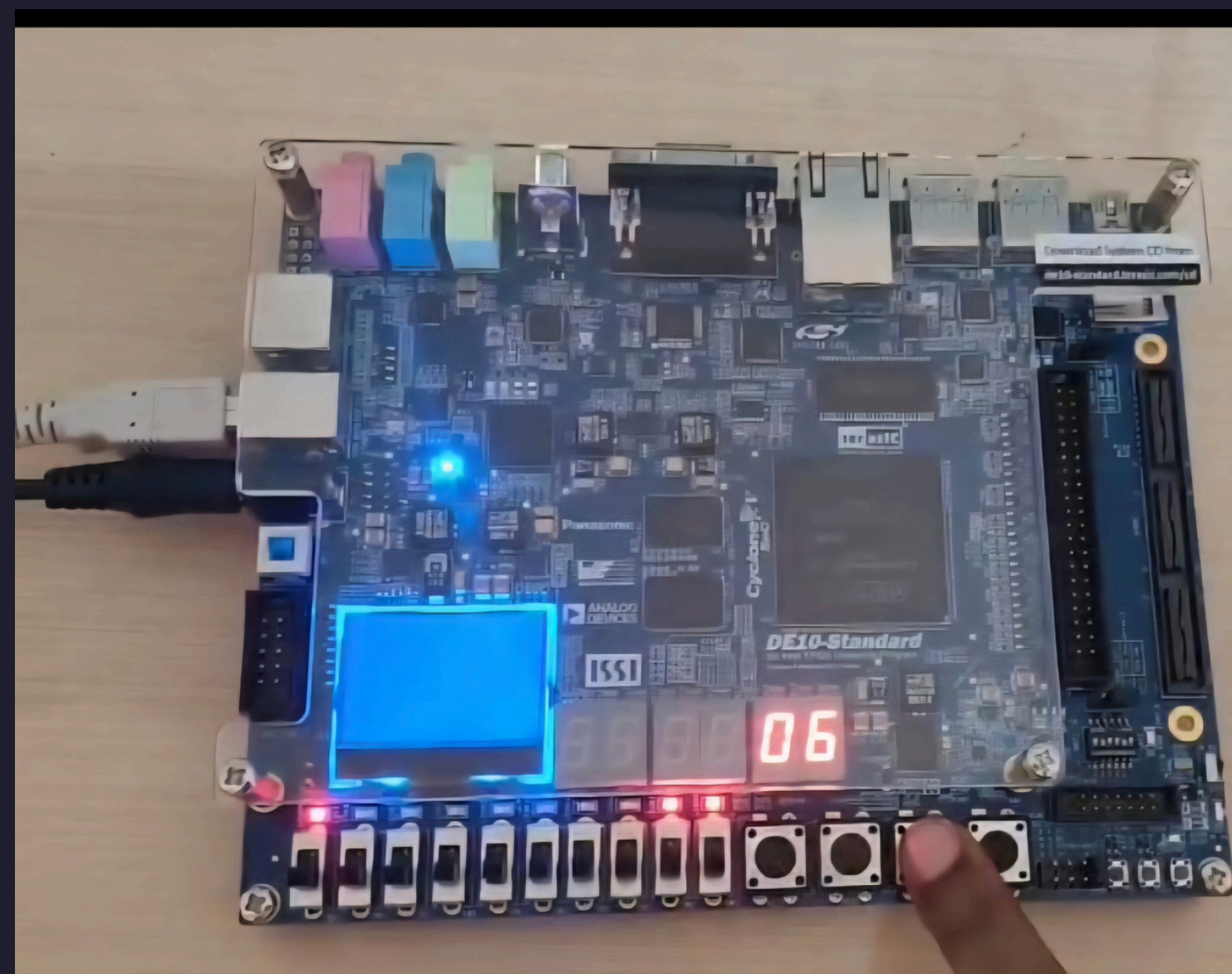
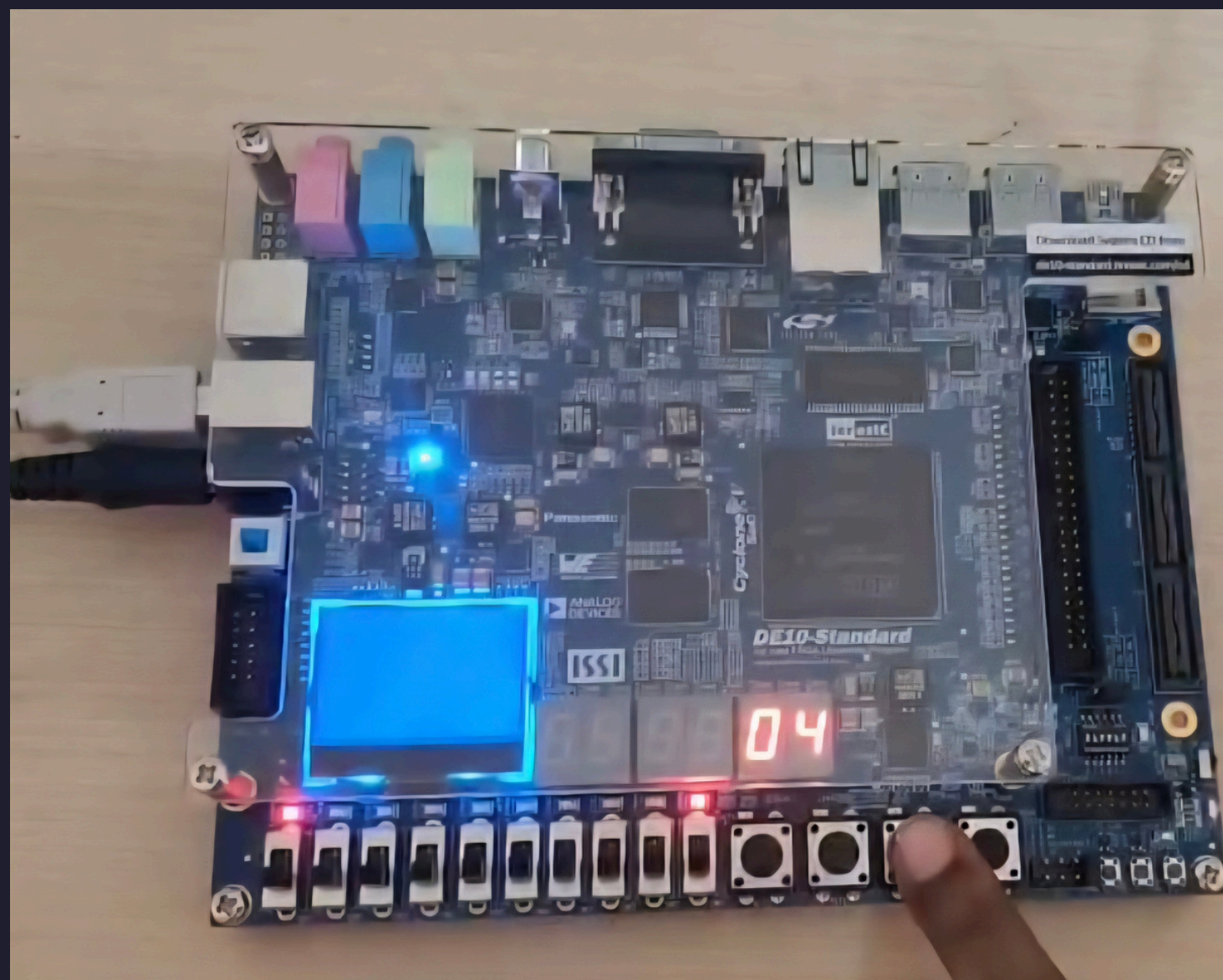
// =====
// 10. TESTBENCH (for simulation only)
// =====
module tb_risc_processor;
    reg clk, reset;
    wire [7:0] pc, acc;
    wire halt_flag;
    risc_processor_8bit uut (.clk(clk), .reset(reset), .pc_out(pc), .acc_out(acc),
        .halt_flag(halt_flag));
    initial begin clk = 0; forever #5 clk = ~clk; end
    initial begin
        $display("---- Starting Simulation ----");
        reset = 1; #15; reset = 0;
        wait(halt_flag == 1); #50;
        $display("PC=%h ACC=%h", pc, acc);
        $stop;
    end
endmodule

```

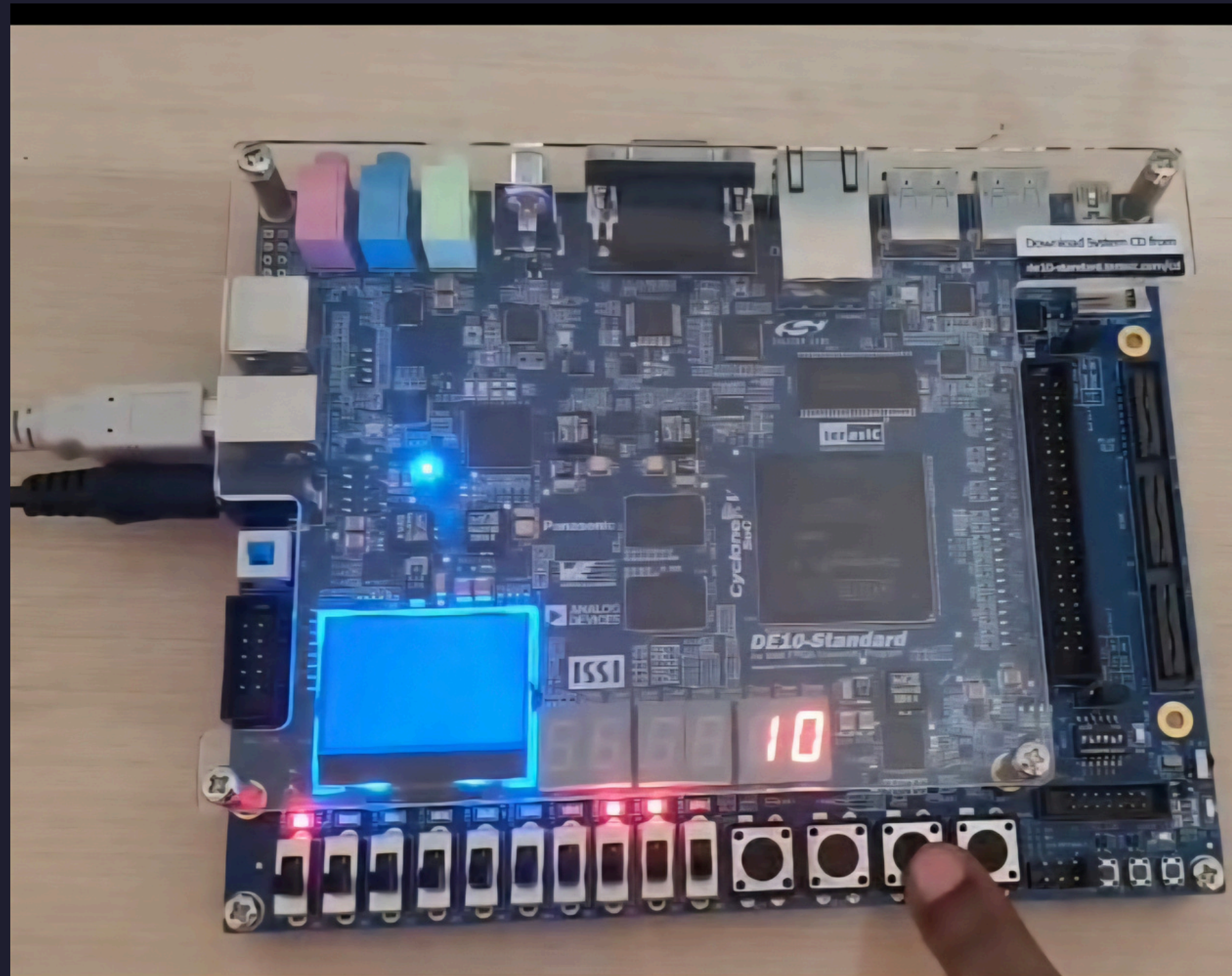
MODELSIM SIMULATION OUTPUT



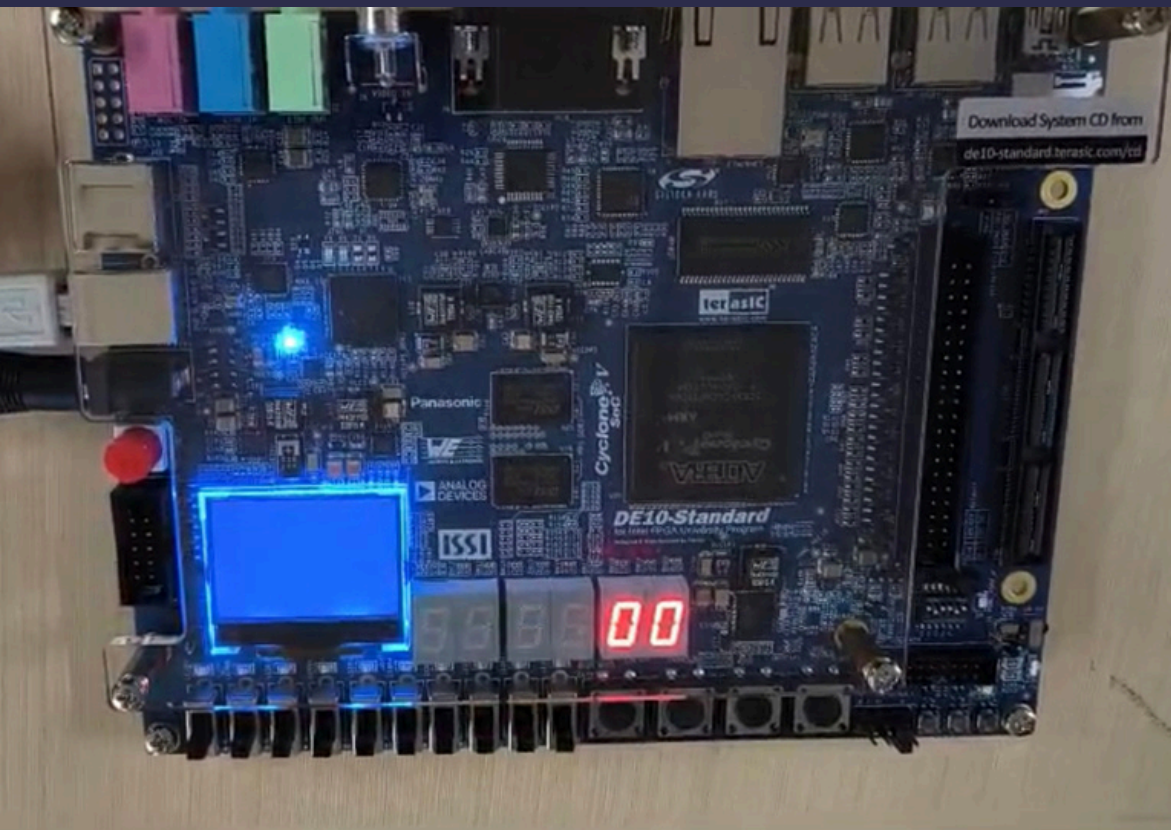
FPGA INPUT ADDITION



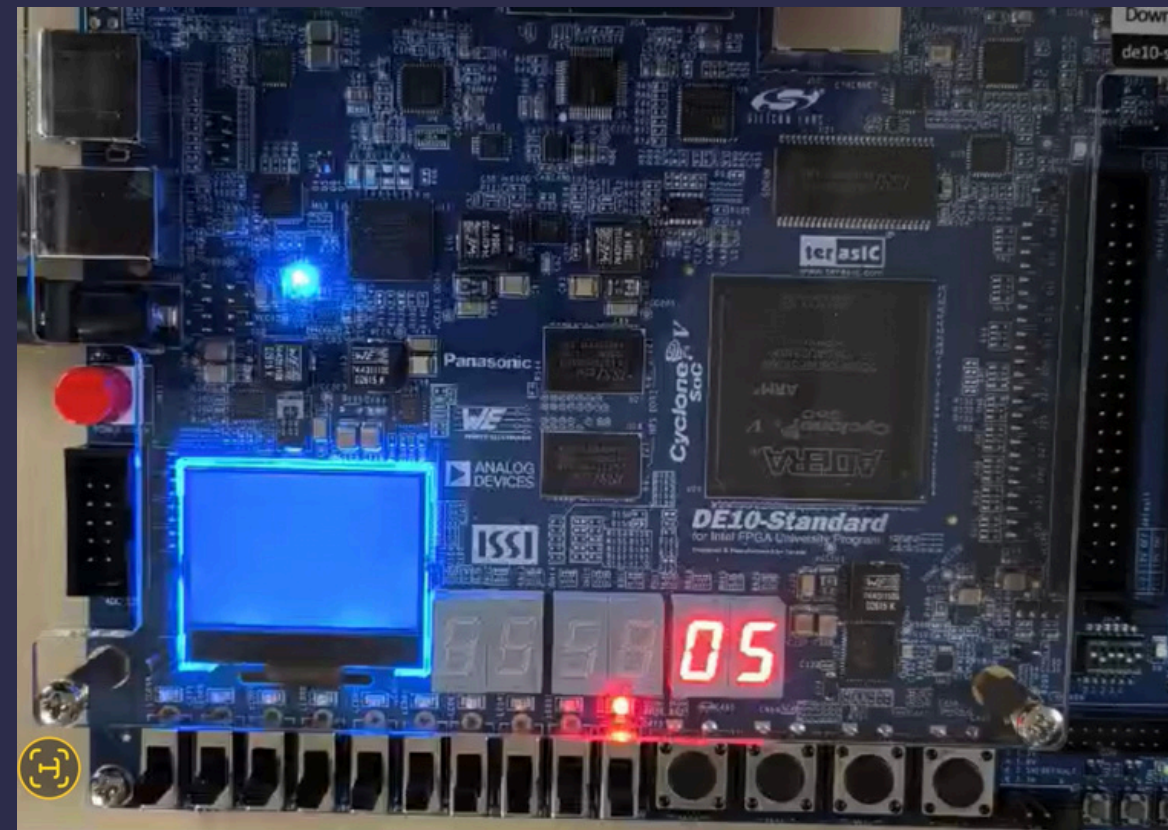
FPGA OUTPUT ADDITION



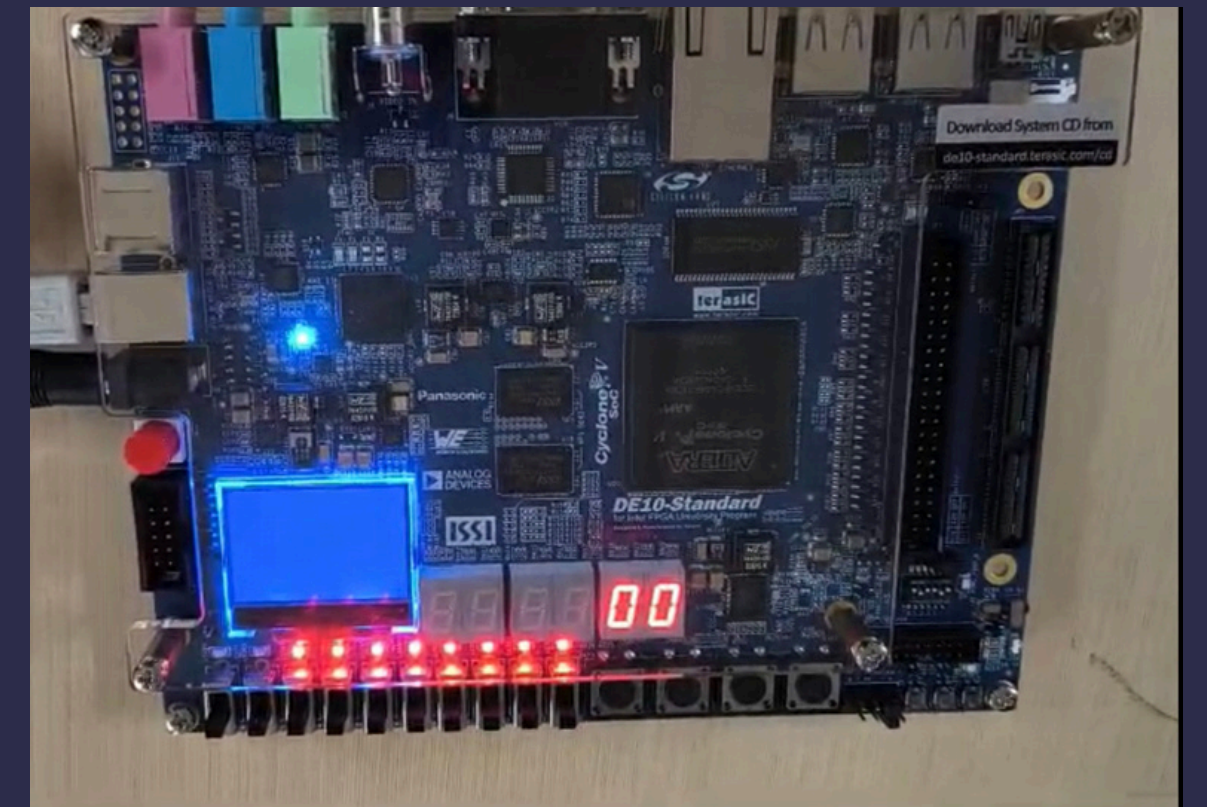
FPGA COUNTER



0000000000
DECIMAL 0



0000000001
DECIMAL 1



1111111111
DECIMAL 255

VERILOG CODE FOR SUBTRACTION

```
// =====  
// 2. INSTRUCTION MEMORY MODULE  
// =====  
module instruction_memory(  
    input wire [7:0] address,  
    output reg [7:0] instruction  
);  
    reg [7:0] mem [0:255];  
    integer i;  
    initial begin  
        // Program: Display 10, then 4, then 10-4=6  
        // Step 1: Load 10 into R0, HALT to display  
        mem[8'h00] = {`LDI, 4'hA}; // R0 = 10 (0xA in hex)  
        mem[8'h01] = {`HALT, 4'h0}; // Halt to display 10  
  
        // Step 2: Load 4 into R0, HALT to display  
        mem[8'h02] = {`LDI, 4'h4}; // R0 = 4  
        mem[8'h03] = {`HALT, 4'h0}; // Halt to display 4  
  
        // Step 3: Load 10, SUB 4 from R2, HALT to display result  
        mem[8'h04] = {`LDI, 4'hA}; // R0 = 10  
        mem[8'h05] = {`SUB, 4'h2}; // R0 = R0 - R2 (R2=4, so R0=6)  
        mem[8'h06] = {`HALT, 4'h0}; // Halt to display 6  
  
        for (i = 7; i < 256; i = i + 1)  
            mem[i] = 8'h00;  
    end  
    always @(*) begin  
        instruction = mem[address];  
    end  
endmodule
```

```
// =====  
// 11. TESTBENCH (for simulation only)  
// =====  
module tb_risc_processor;  
    reg clk, reset, resume;  
    wire [7:0] pc, acc;  
    wire halt_flag;  
  
    risc_processor_8bit uut (  
        .clk(clk),  
        .reset(reset),  
        .resume(resume),  
        .pc_out(pc),  
        .acc_out(acc),  
        .halt_flag(halt_flag)  
    );  
  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    initial begin  
        $display("--- Starting Simulation ---");  
        reset = 1; resume = 0; #15; reset = 0;  
  
        // Wait for first HALT (display 4)  
        wait(halt_flag == 1);  
        $display("Step 1: PC=%h ACC=%d", pc, acc);  
        #50;  
  
        // Resume to display 6  
        resume = 1; #10; resume = 0;  
    end  
endmodule
```

VERILOG CODE FOR SUBTRACTION

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

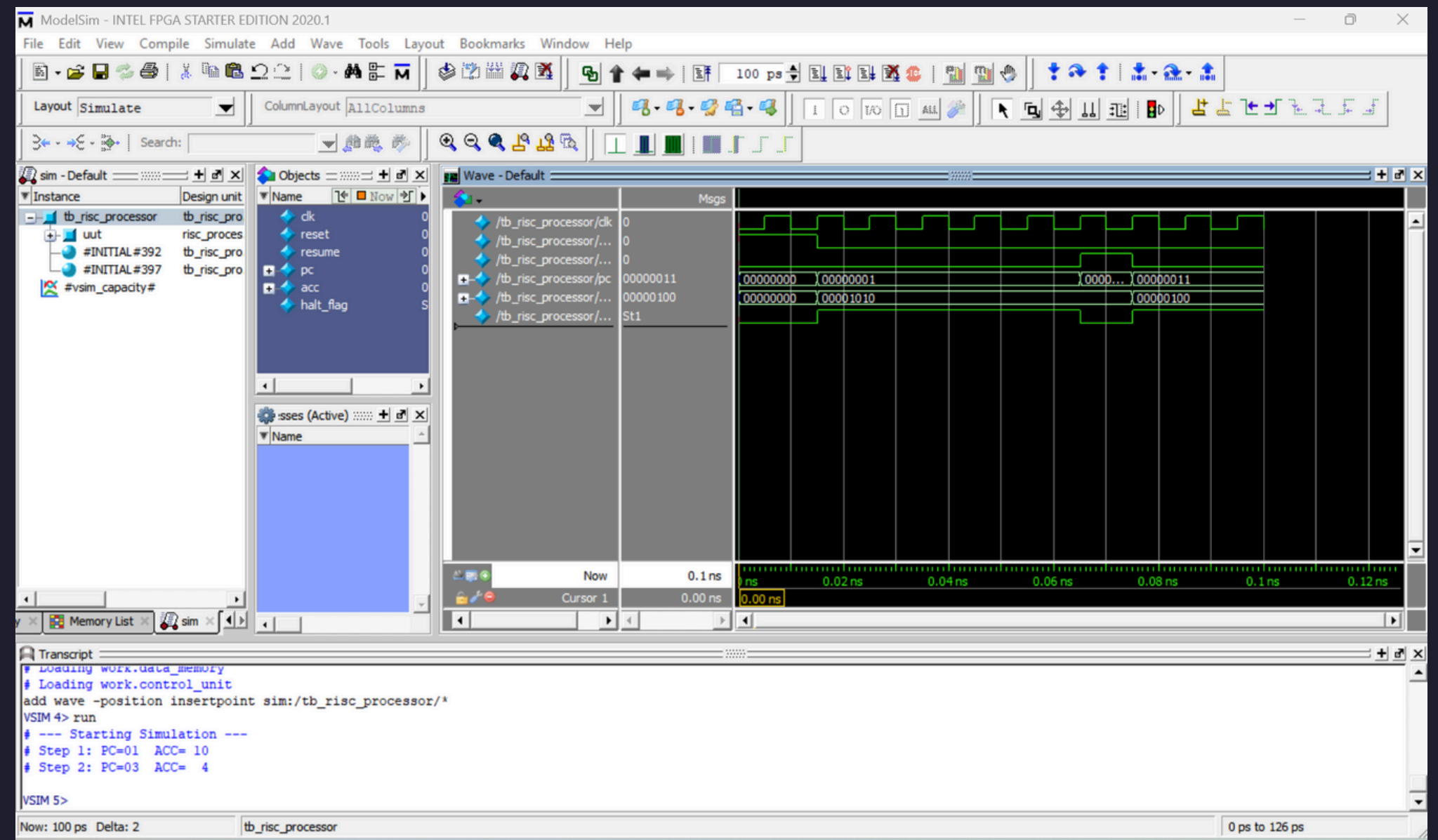
initial begin
    $display("--- Starting simulation ---");
    reset = 1; resume = 0; #15; reset = 0;

    // Wait for first HALT (display 4)
    wait(halt_flag == 1);
    $display("Step 1: PC=%h ACC=%d", pc, acc);
    #50;

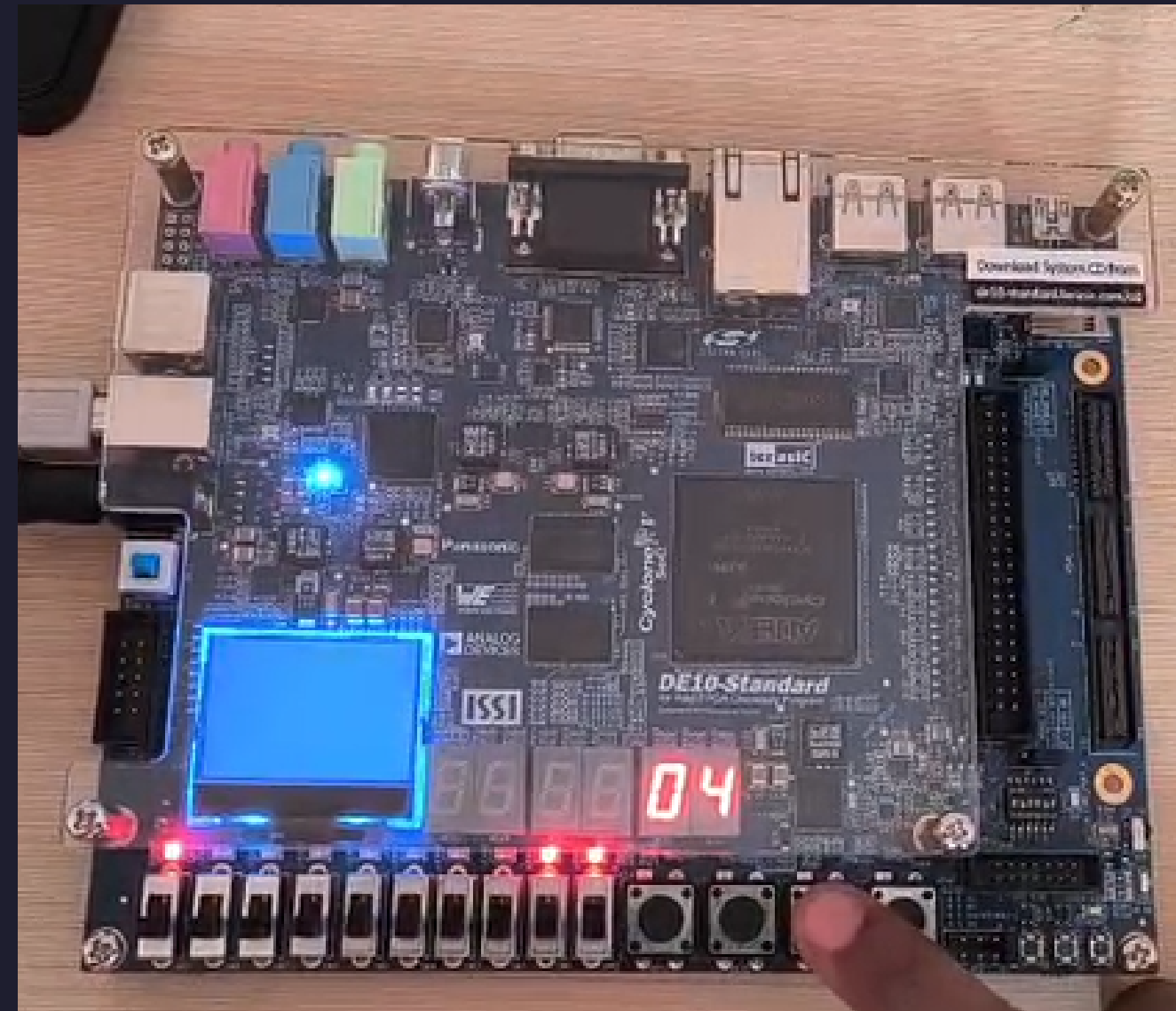
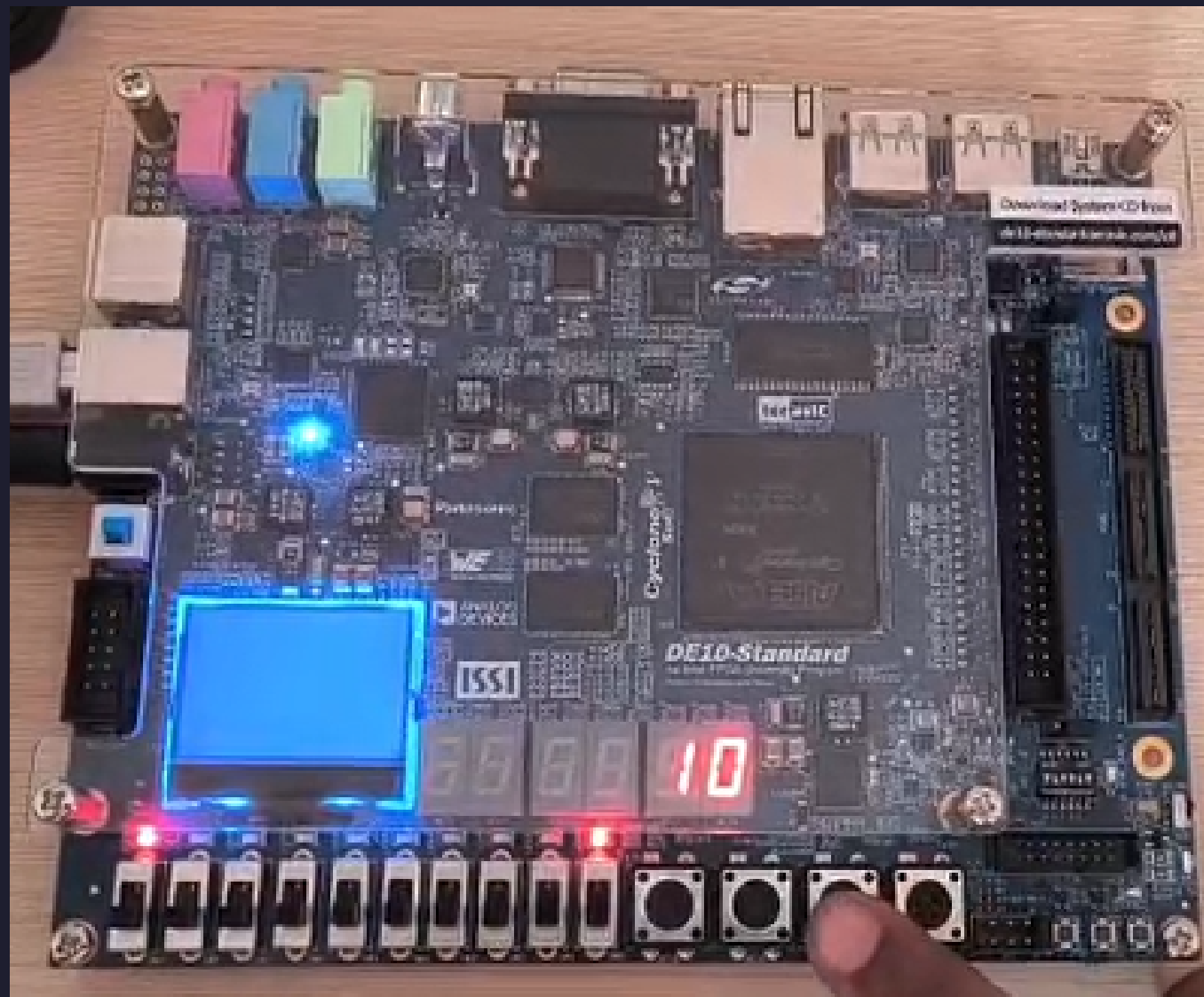
    // Resume to display 6
    resume = 1; #10; resume = 0;
    wait(halt_flag == 1);
    $display("Step 2: PC=%h ACC=%d", pc, acc);
    #50;

    // Resume to display 10
    resume = 1; #10; resume = 0;
    wait(halt_flag == 1);
    $display("Step 3: PC=%h ACC=%d (4+6)", pc, acc);
    #50;

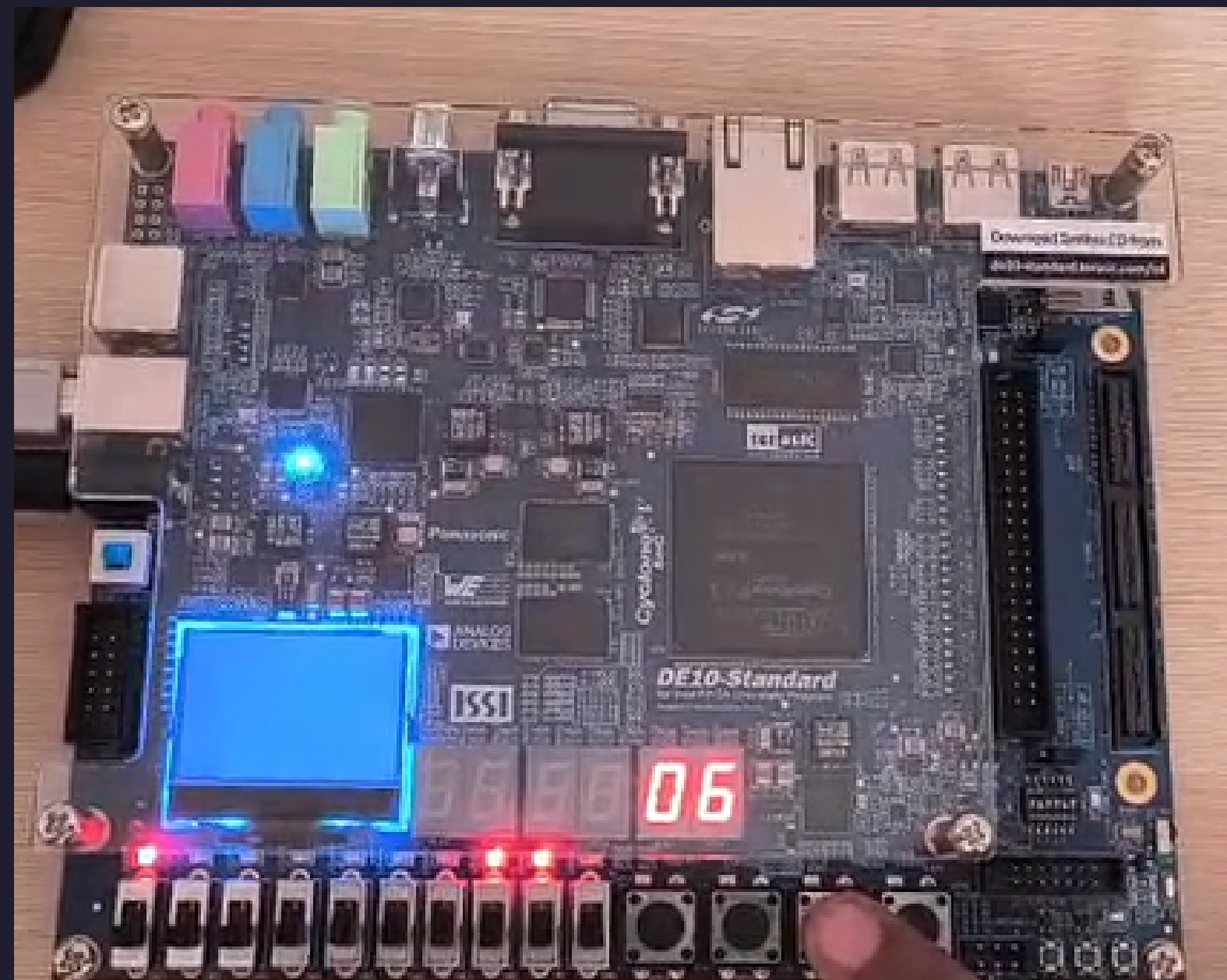
    $stop;
end
endmodule
```



FPGA INPUT SUBTRACTION



FPGA OUTPUT SUBTRACTION



VERILOG CODE FOR OR OPERATION

```
// =====  
// 2. INSTRUCTION MEMORY MODULE  
// =====  
module instruction_memory(  
    input wire [7:0] address,  
    output reg [7:0] instruction  
);  
    reg [7:0] mem [0:255];  
    integer i;  
    initial begin  
        // Program: Display 12, then 10, then 12 OR 10 = 14  
        // Step 1: Load 12 into R0, HALT to display  
        mem[8'h00] = {`LDI, 4'hC}; // R0 = 12 (0xC in hex = 1100 binary)  
        mem[8'h01] = {`HALT, 4'h0}; // Halt to display 12  
  
        // Step 2: Load 10 into R0, HALT to display  
        mem[8'h02] = {`LDI, 4'hA}; // R0 = 10 (0xA in hex = 1010 binary)  
        mem[8'h03] = {`HALT, 4'h0}; // Halt to display 10  
  
        // Step 3: Load 12, OR with 10 from R1, HALT to display result  
        mem[8'h04] = {`LDI, 4'hC}; // R0 = 12 (1100 binary)  
        mem[8'h05] = {`OR, 4'h1}; // R0 = R0 OR R1 (R1=10=1010, result=1110=14)  
        mem[8'h06] = {`HALT, 4'h0}; // Halt to display 14  
  
        for (i = 7; i < 256; i = i + 1)  
            mem[i] = 8'h00;  
    end  
  
    always @(*) begin  
        instruction = mem[address];  
    end  
endmodule
```

```
// =====  
// 11. TESTBENCH (for simulation only)  
// =====  
module tb_risc_processor;  
    reg clk, reset, resume;  
    wire [7:0] pc, acc;  
    wire halt_flag;  
  
    risc_processor_8bit uut (  
        .clk(clk),  
        .reset(reset),  
        .resume(resume),  
        .pc_out(pc),  
        .acc_out(acc),  
        .halt_flag(halt_flag)  
    );  
  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    initial begin  
        $display("--- Starting simulation ---");  
        reset = 1; resume = 0; #15; reset = 0;  
  
        // wait for first HALT (display 4)  
        wait(halt_flag == 1);  
        $display("Step 1: PC=%h ACC=%d", pc, acc);  
        #50;  
  
        // Resume to display 6  
        resume = 1; #10; resume = 0;  
    end  
endmodule
```

VERILOG CODE FOR OR OPERATION

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

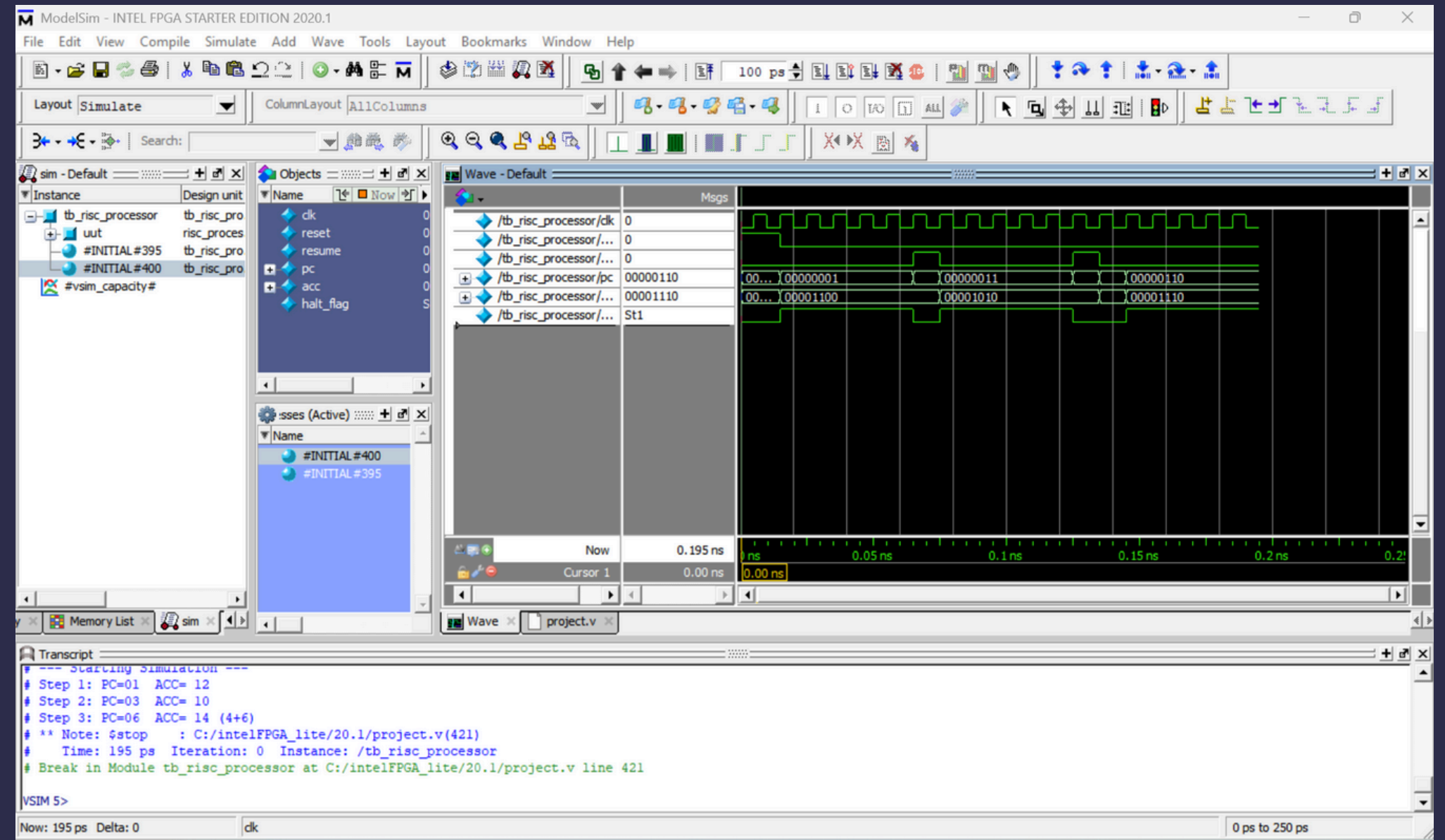
initial begin
    $display("--- Starting simulation ---");
    reset = 1; resume = 0; #15; reset = 0;

    // Wait for first HALT (display 4)
    wait(halt_flag == 1);
    $display("Step 1: PC=%h ACC=%d", pc, acc);
    #50;

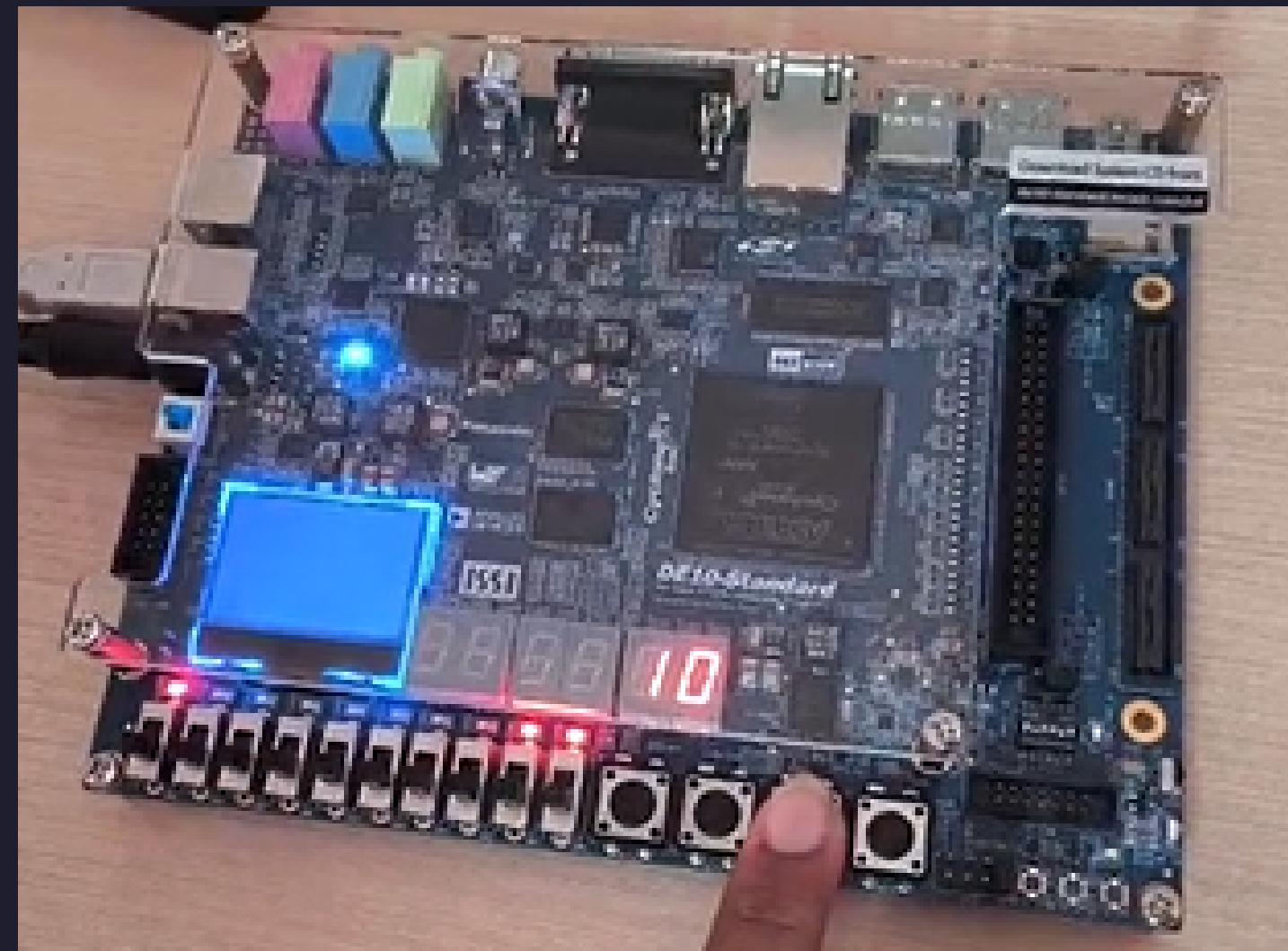
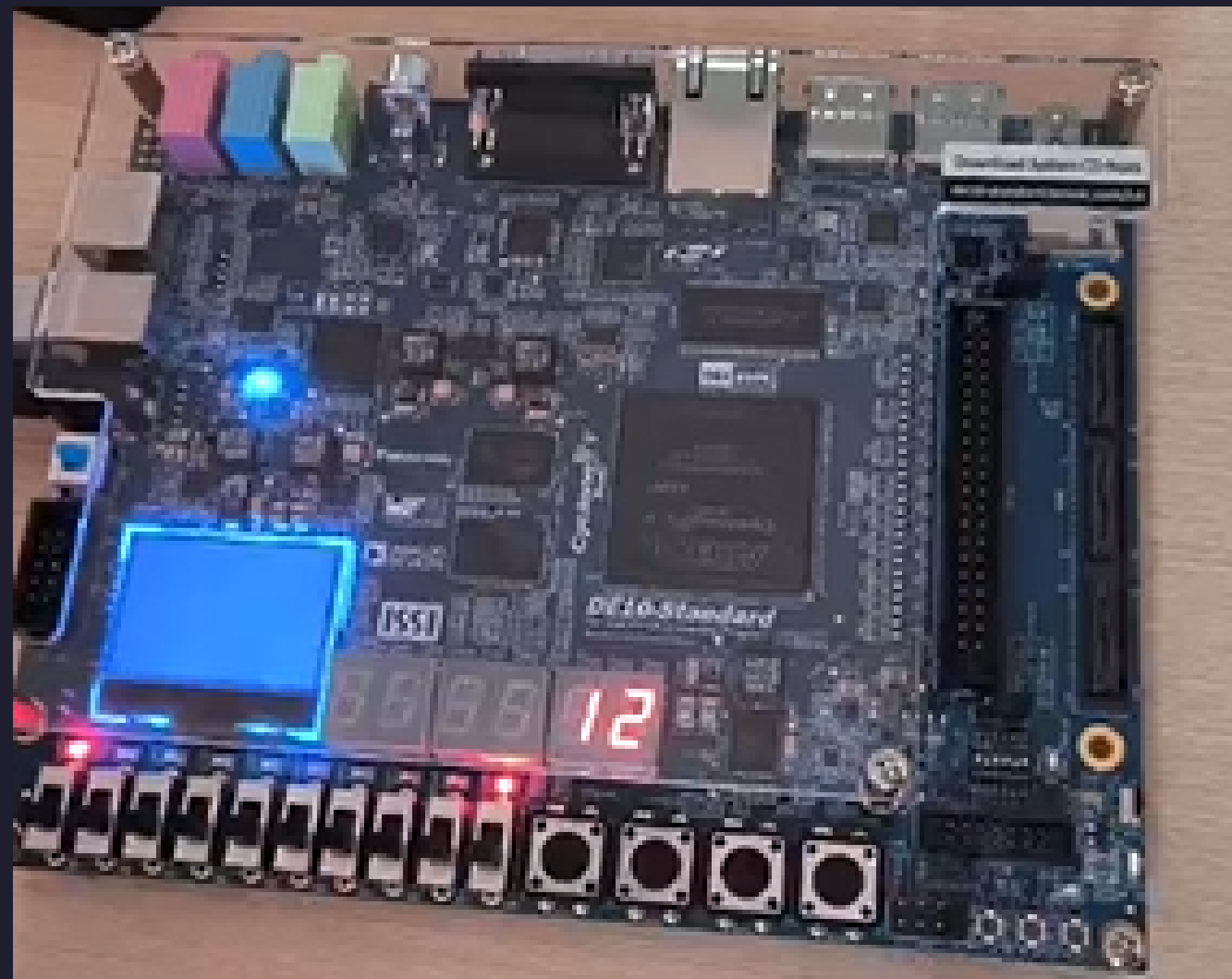
    // Resume to display 6
    resume = 1; #10; resume = 0;
    wait(halt_flag == 1);
    $display("Step 2: PC=%h ACC=%d", pc, acc);
    #50;

    // Resume to display 10
    resume = 1; #10; resume = 0;
    wait(halt_flag == 1);
    $display("Step 3: PC=%h ACC=%d (4+6)", pc, acc);
    #50;

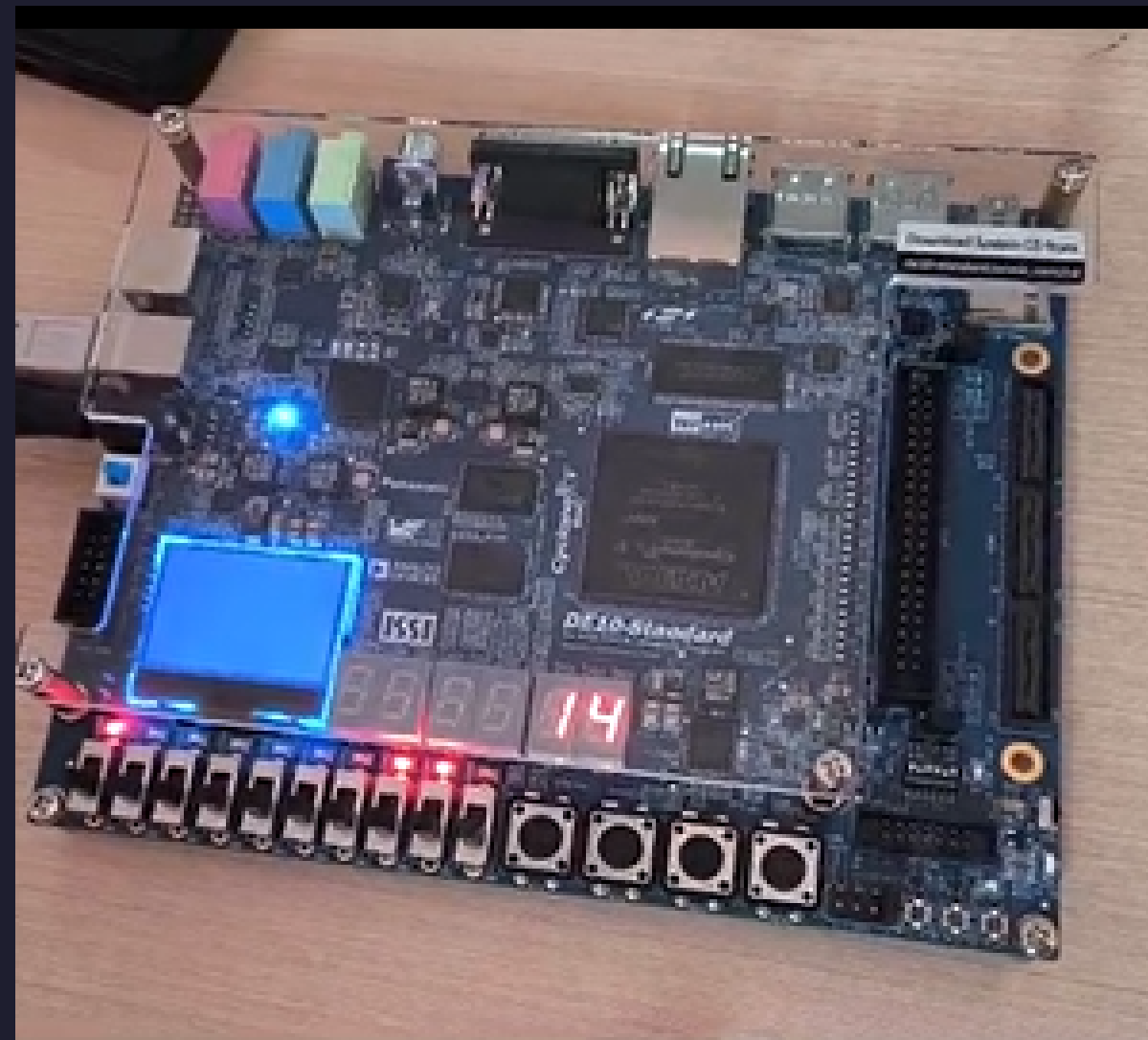
    $stop;
end
endmodule
```



FPGA INPUT OR GATE



FPGA OUTPUT OR GATE

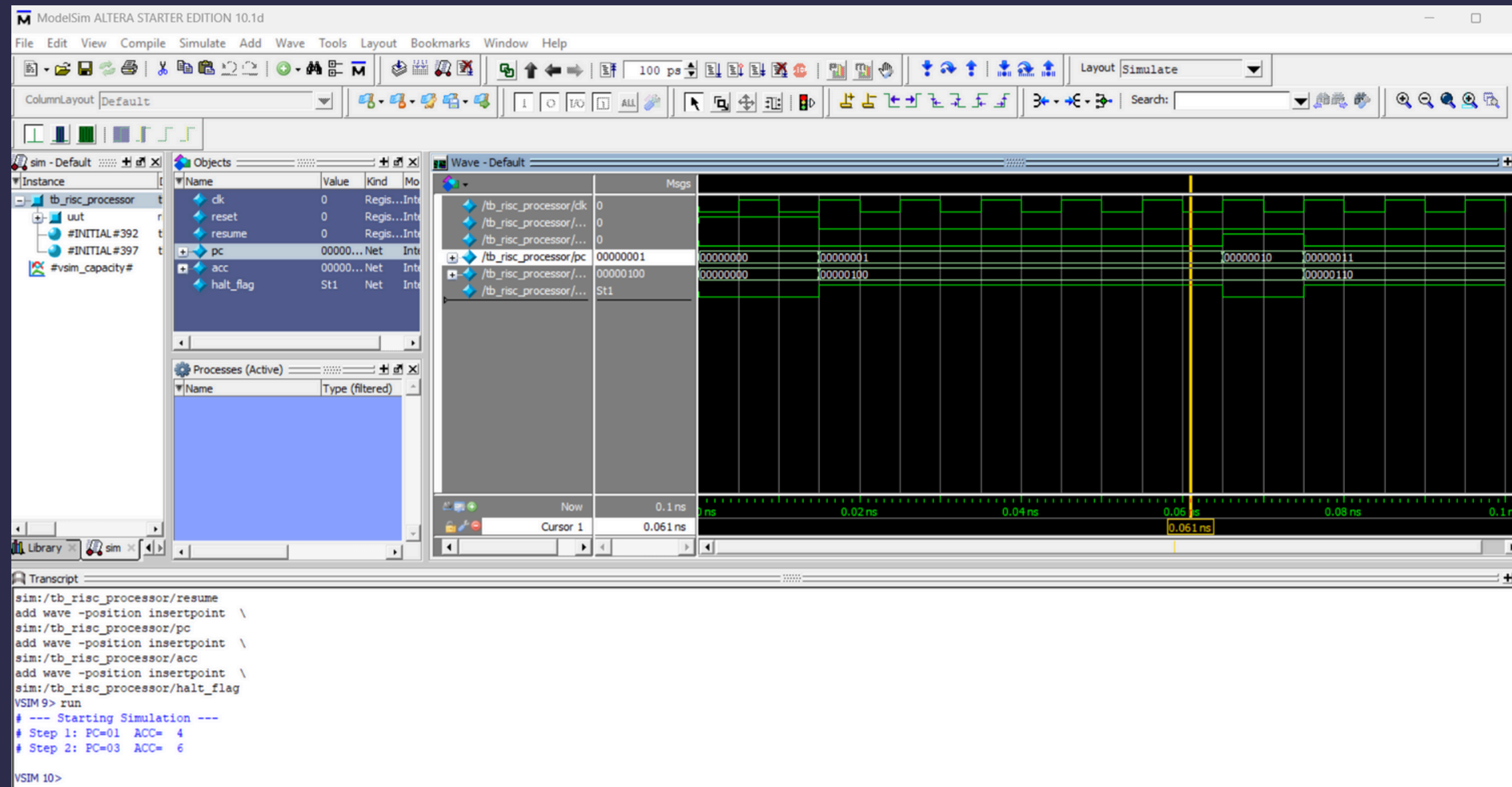


VERILOG CODE FOR AND OPERATION

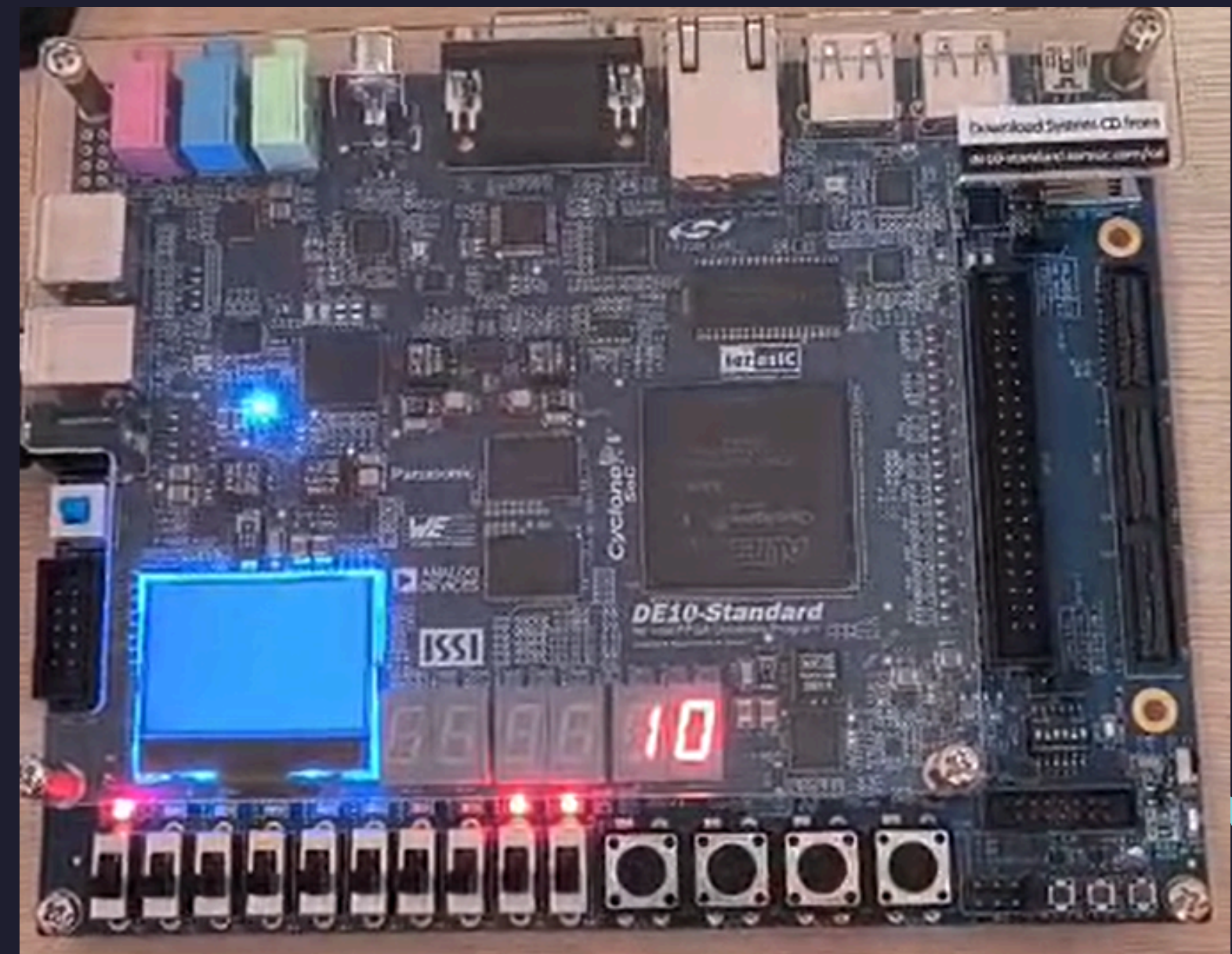
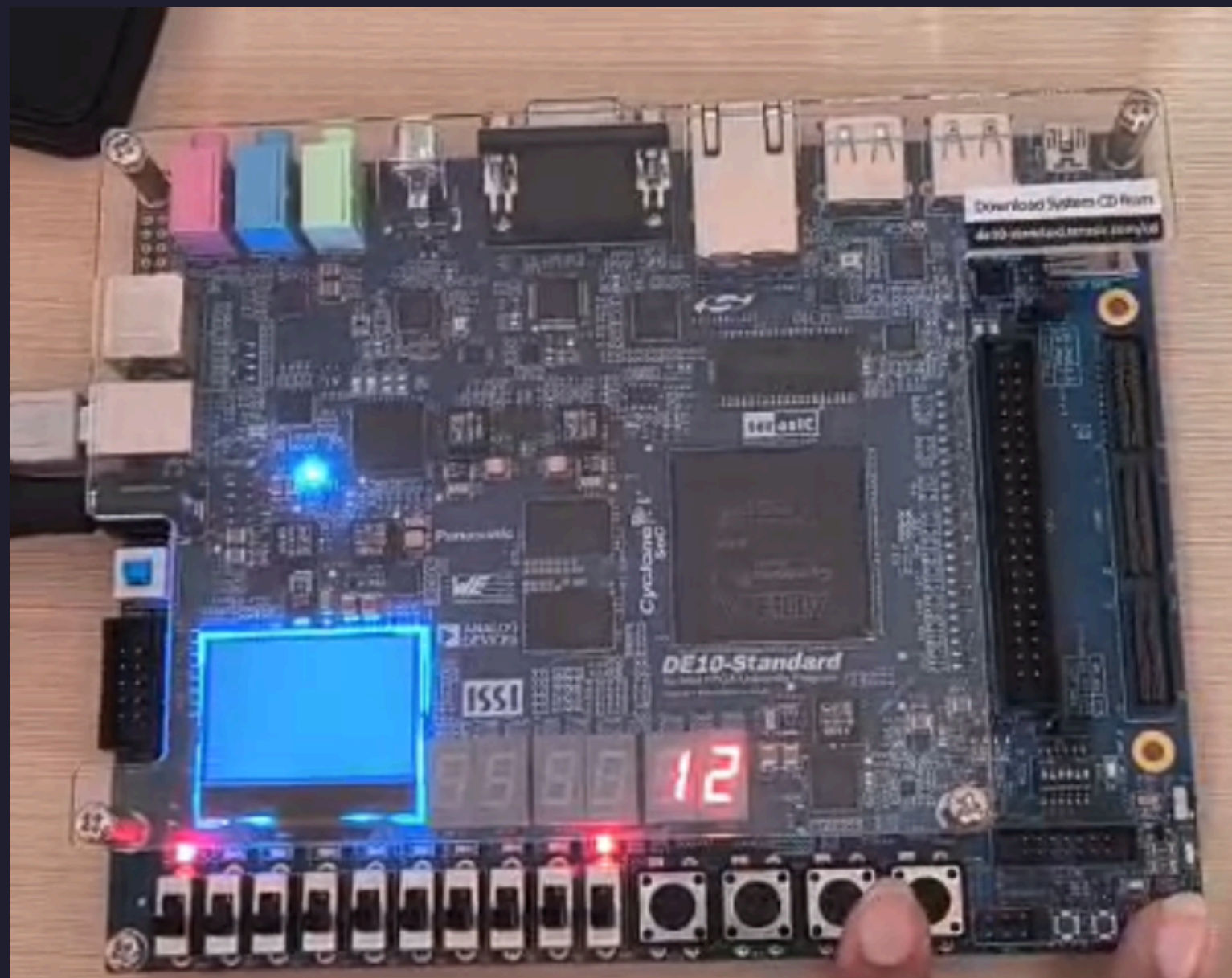
```
// =====  
// 2. INSTRUCTION MEMORY MODULE  
// =====  
module instruction_memory(  
    input wire [7:0] address,  
    output reg [7:0] instruction  
);  
    reg [7:0] mem [0:255];  
    integer i;  
    initial begin  
        // Program: Display 12, then 10, then 12 AND 10 = 8  
        // Step 1: Load 12 into R0, HALT to display  
        mem[8'h00] = {\LDI, 4'hC}; // R0 = 12 (0xC in hex = 1100 binary)  
        mem[8'h01] = {\HALT, 4'h0}; // Halt to display 12  
  
        // Step 2: Load 10 into R0, HALT to display  
        mem[8'h02] = {\LDI, 4'hA}; // R0 = 10 (0xA in hex = 1010 binary)  
        mem[8'h03] = {\HALT, 4'h0}; // Halt to display 10  
  
        // Step 3: Load 12, AND with 10 from R1, HALT to display result  
        mem[8'h04] = {\LDI, 4'hC}; // R0 = 12 (1100 binary)  
        mem[8'h05] = {\AND, 4'h1}; // R0 = R0 AND R1 (R1=10=1010, result=1000=8)  
        mem[8'h06] = {\HALT, 4'h0}; // Halt to display 8  
  
        for (i = 7; i < 256; i = i + 1)  
            mem[i] = 8'h00;  
    end  
  
    always @(*) begin  
        instruction = mem[address];  
    end  
endmodule
```

```
module tb_risc_processor;  
    reg clk, reset, resume;  
    wire [7:0] pc, acc;  
    wire halt_flag;  
  
    risc_processor_8bit uut (  
        .clk(clk),  
        .reset(reset),  
        .resume(resume),  
        .pc_out(pc),  
        .acc_out(acc),  
        .halt_flag(halt_flag)  
    );  
  
    initial begin  
        $display("--- Starting Simulation ---");  
        reset = 1; resume = 0; #15; reset = 0;  
  
        // Wait for first HALT (display 4)  
        wait(halt_flag == 1);  
        $display("Step 1: PC=%h ACC=%d", pc, acc);  
        #50;  
  
        // Resume to display 6  
        resume = 1; #10; resume = 0;  
        wait(halt_flag == 1);  
        $display("Step 2: PC=%h ACC=%d", pc, acc);  
        #50;  
  
        // Resume to display 10  
        resume = 1; #10; resume = 0;  
        wait(halt_flag == 1);  
        $display("Step 3: PC=%h ACC=%d (4+6)", pc, acc);  
        #50;  
  
        $stop;  
    end  
endmodule
```

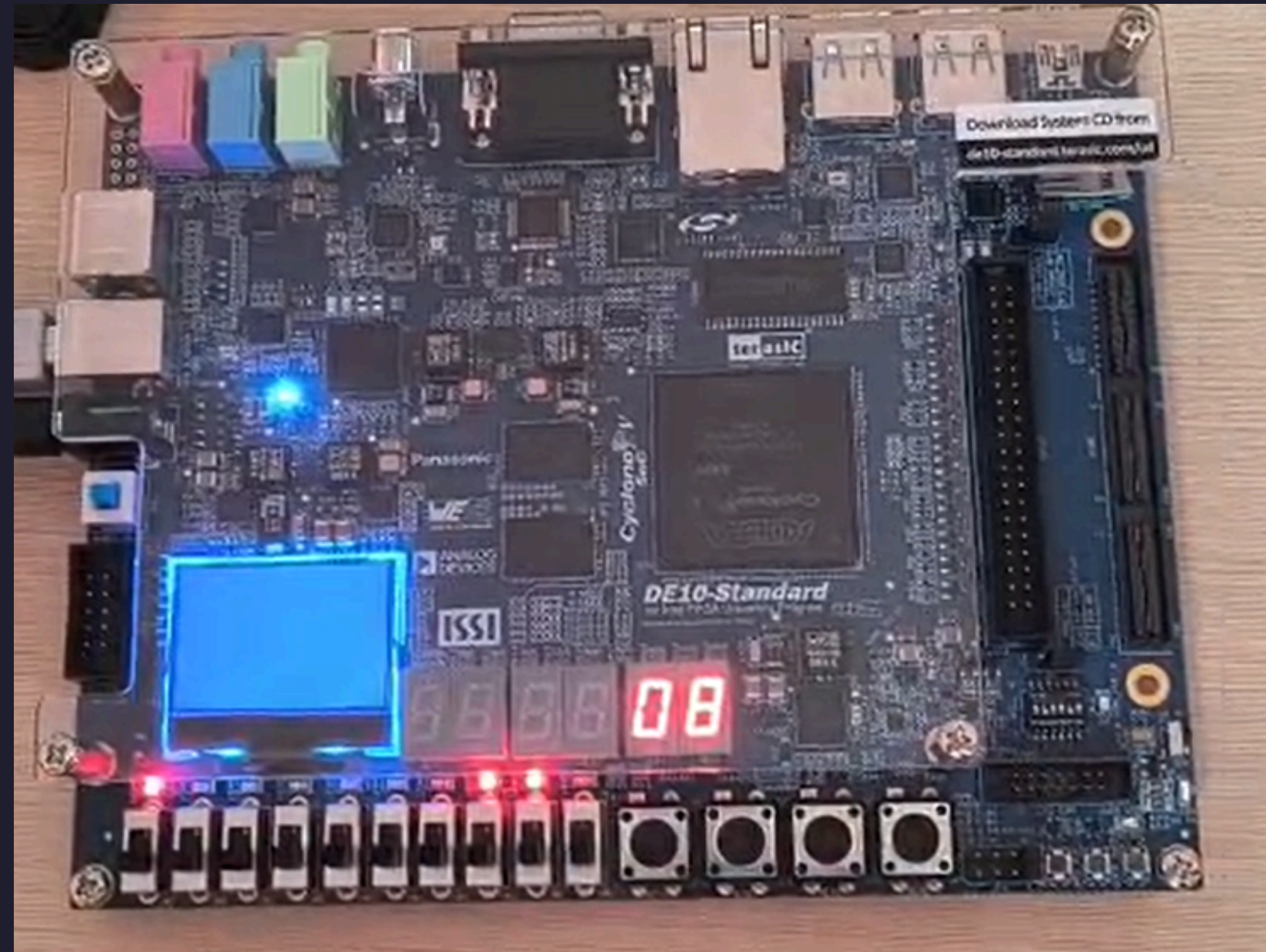
MODELSIM OUTPUT



FPGA INPUT AND GATE



FPGA OUTPUT AND GATE



VERILOG CODE FOR XOR OPERATION

```
// =====  
// 2. INSTRUCTION MEMORY MODULE  
// =====  
module instruction_memory(  
    input wire [7:0] address,  
    output reg [7:0] instruction  
);  
    reg [7:0] mem [0:255];  
    integer i;  
    initial begin  
        // Program: Display 12, then 10, then 12 XOR 10 = 6  
        // Step 1: Load 12 into R0, HALT to display  
        mem[8'h00] = {`LDI, 4'hc}; // R0 = 12 (0xC in hex = 1100 binary)  
        mem[8'h01] = {`HALT, 4'h0}; // Halt to display 12  
  
        // Step 2: Load 10 into R0, HALT to display  
        mem[8'h02] = {`LDI, 4'ha}; // R0 = 10 (0xA in hex = 1010 binary)  
        mem[8'h03] = {`HALT, 4'h0}; // Halt to display 10  
  
        // Step 3: Load 12, XOR with 10 from R1, HALT to display result  
        mem[8'h04] = {`LDI, 4'hc}; // R0 = 12 (1100 binary)  
        mem[8'h05] = {`XOR, 4'h1}; // R0 = R0 XOR R1 (R1=10=1010, result=0110=6)  
        mem[8'h06] = {`HALT, 4'h0}; // Halt to display 6  
  
        for (i = 7; i < 256; i = i + 1)  
            mem[i] = 8'h00;  
    end  
    always @(*) begin  
        instruction = mem[address];  
    end  
endmodule
```

```
// =====  
// 11. TESTBENCH (for simulation only)  
// =====  
module tb_risc_processor;  
    reg clk, reset, resume;  
    wire [7:0] pc, acc;  
    wire halt_flag;  
  
    risc_processor_8bit uut (  
        .clk(clk),  
        .reset(reset),  
        .resume(resume),  
        .pc_out(pc),  
        .acc_out(acc),  
        .halt_flag(halt_flag)  
    );  
  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    initial begin  
        $display("--- Starting simulation ---");  
        reset = 1; resume = 0; #15; reset = 0;  
  
        // Wait for first HALT (display 4)  
        wait(halt_flag == 1);  
        $display("Step 1: PC=%h ACC=%d", pc, acc);  
        #50;  
  
        // Resume to display 6  
        resume = 1; #10; resume = 0;  
    end  
endmodule
```


VERILOG CODE FOR XOR OPERATION

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

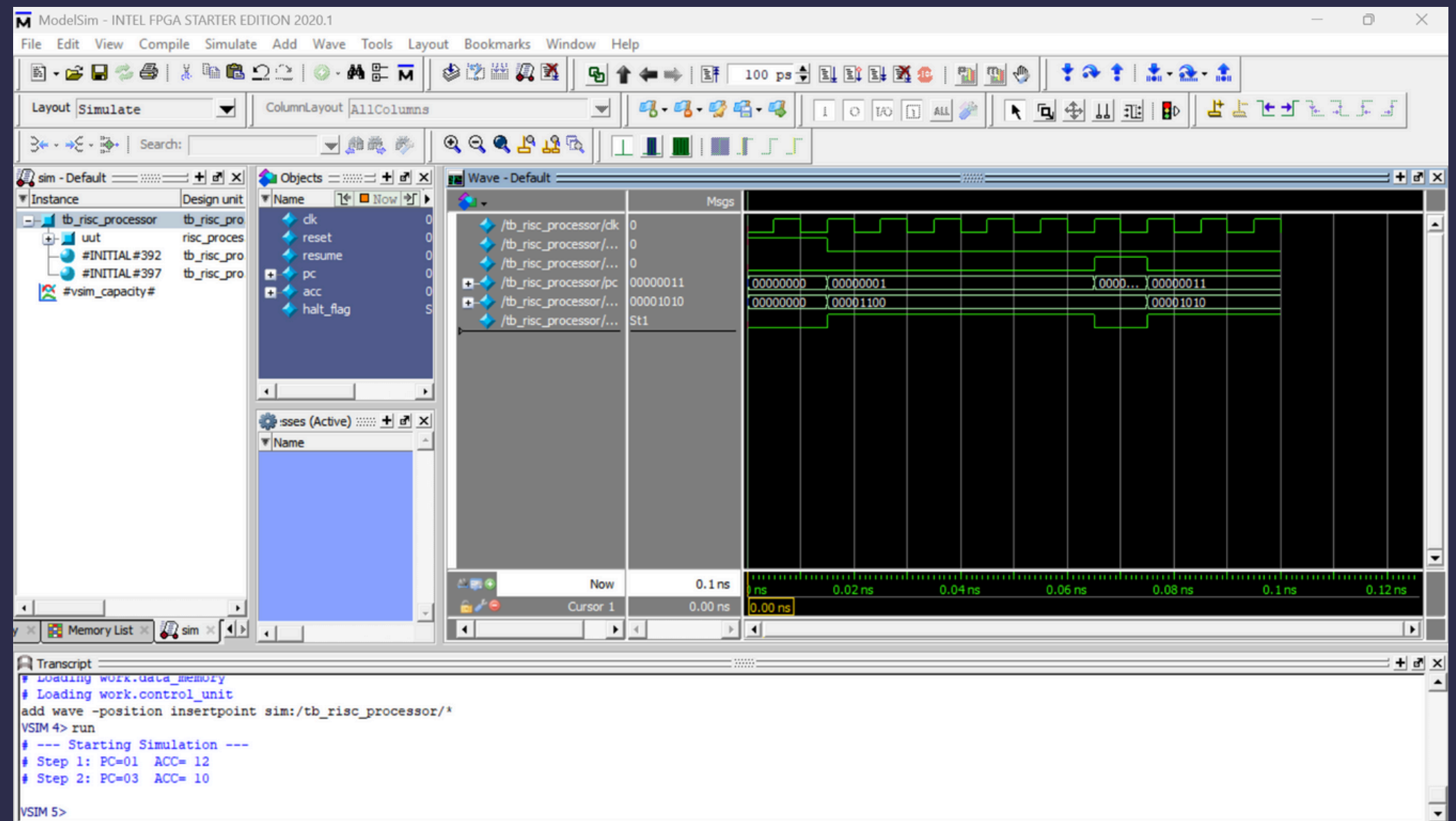
initial begin
    $display("--- Starting Simulation ---");
    reset = 1; resume = 0; #15; reset = 0;

    // Wait for first HALT (display 4)
    wait(halt_flag == 1);
    $display("Step 1: PC=%h  ACC=%d", pc, acc);
    #50;

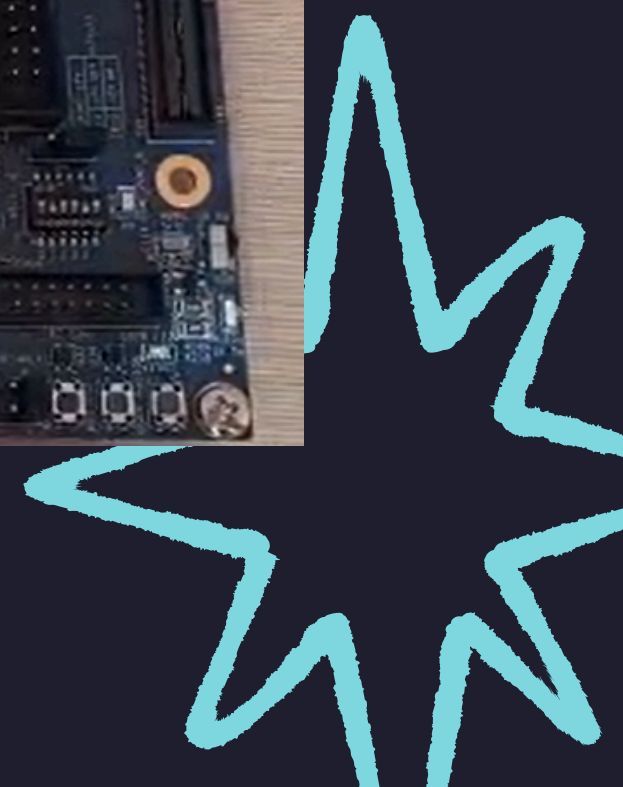
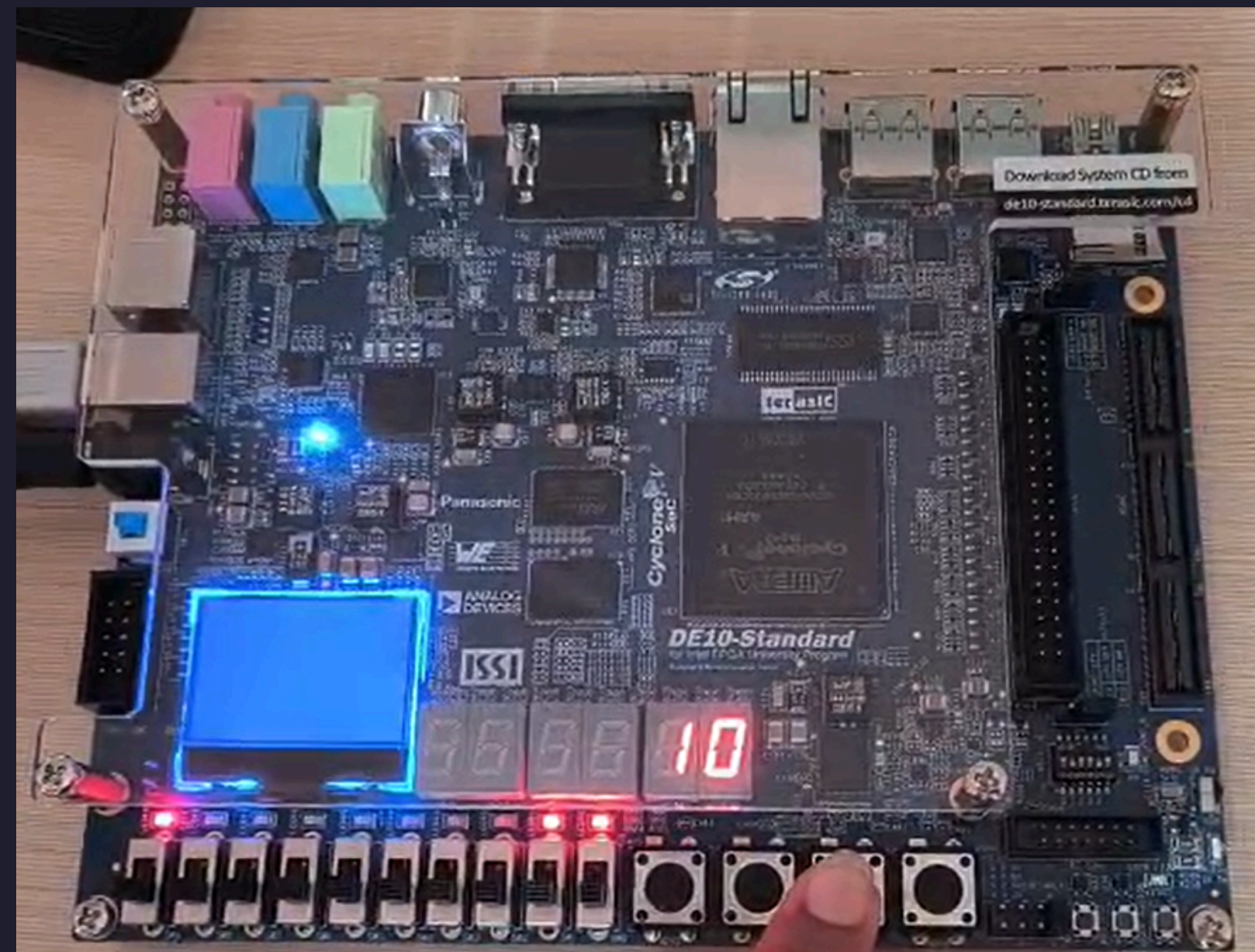
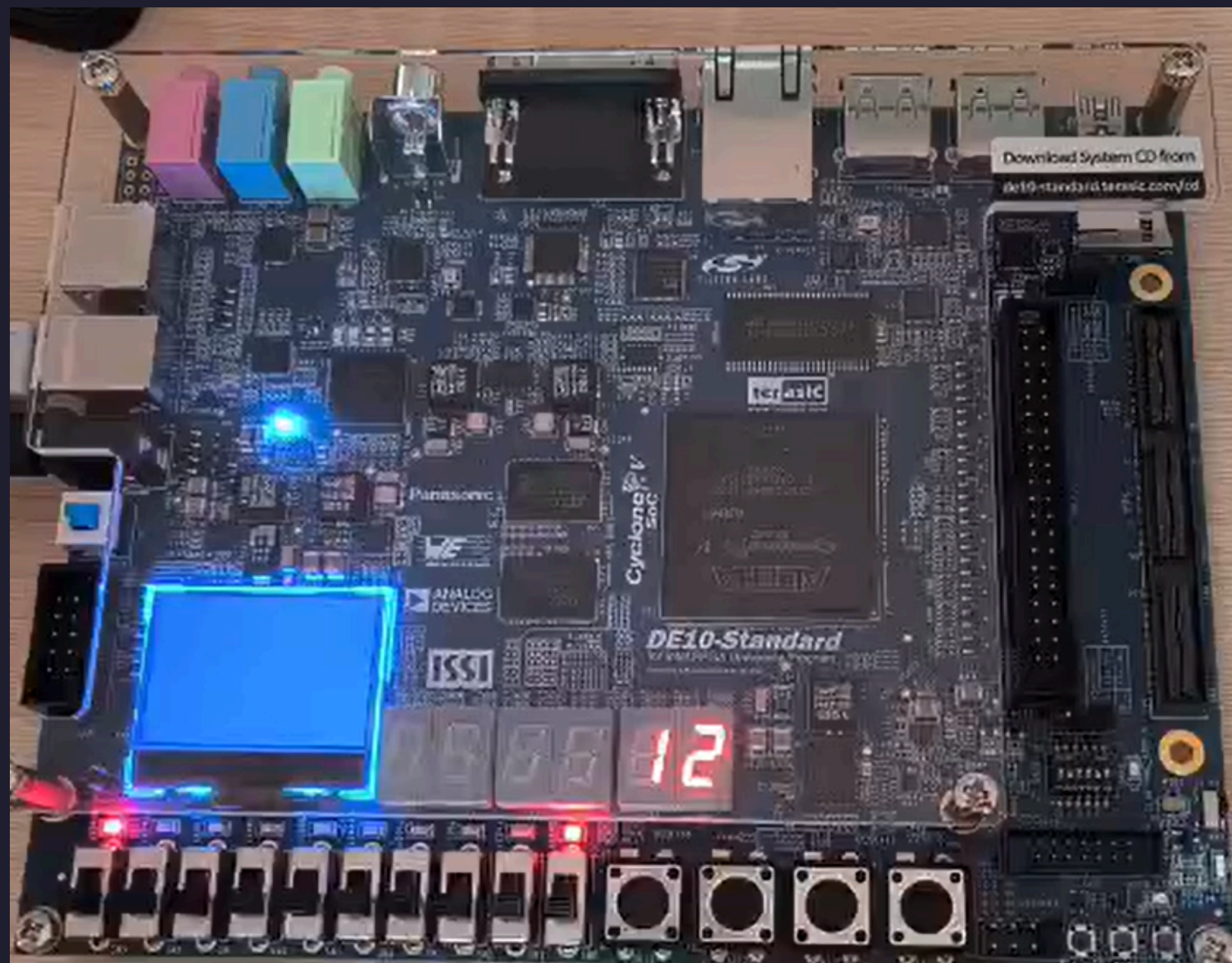
    // Resume to display 6
    resume = 1; #10; resume = 0;
    wait(halt_flag == 1);
    $display("Step 2: PC=%h  ACC=%d", pc, acc);
    #50;

    // Resume to display 10
    resume = 1; #10; resume = 0;
    wait(halt_flag == 1);
    $display("Step 3: PC=%h  ACC=%d (4+6)", pc, acc);
    #50;

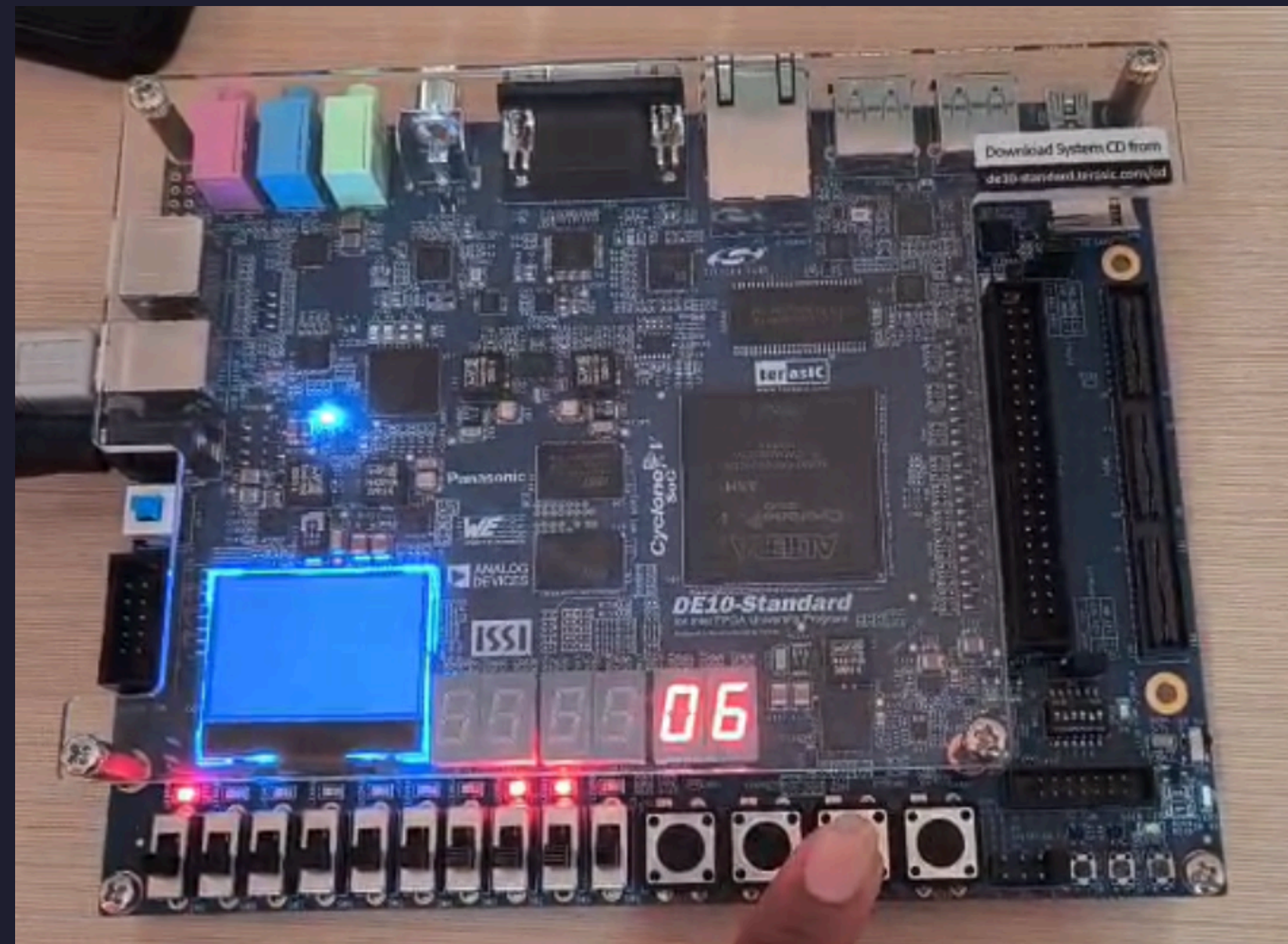
    $stop;
end
endmodule
```

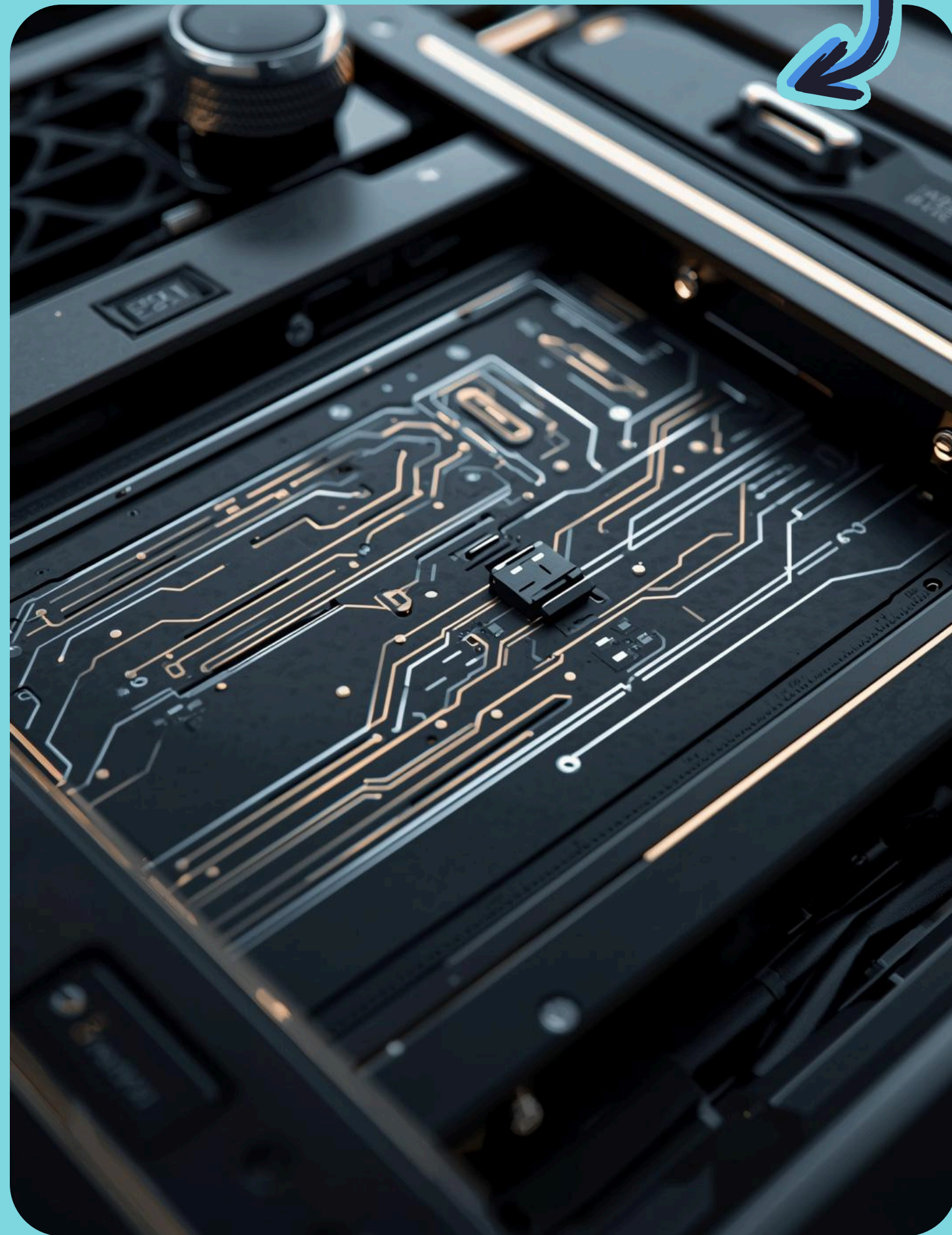


FPGA INPUT XOR GATE



FPGA OUTPUT XOR GATE





REFERENCES

- E. Ayeh, K. Agbedanu, Y. Morita, O. Adamo and P. Guturu, "FPGA Implementation of an 8-bit Simple Processor," 2008 IEEE Region 5 Conference, Kansas City, MO, USA, 2008,
- Hernandez Zavala, A., Camacho Nieto, O., Huerta Ruelas, J. A., & Carvallo Domínguez, A. R. (2015).
- Jeemon, J. (2018). 8-Bit RISC Processor Design using Verilog HDL on FPGA.